Université Ibn Tofaïl — Kénitra
École Nationale des Sciences Appliquées de Kénitra
Filière : Ingénierie des Réseaux et des Systèmes de Télécommunication

AEGIS

Machine Learning-Based Intrusion Detection and Active Defense System

Encadré par

Prof. Aniss Moumen

Réalisé par

Adil Lekhbioui
Khadija Nafia
El Yazid Hammoubel
Moulay Anas
El Kartouti Anas

Année universitaire 2025 – 2026

Contents

List of Figures

List of Tables

General Introduction

The continuous digitalisation of organisations has turned network infrastructures into critical assets shared across LAN, cloud, wireless, and hybrid environments. Protecting these infrastructures is no longer limited to deploying firewalls or signature-based Intrusion Detection Systems; attackers now operate with adaptive, automated tools that probe networks for open services and vulnerabilities before launching targeted intrusions. Port scanning and network reconnaissance constitute the first phase of this kill chain, yet static rule-based defences struggle to keep pace with stealthy, low-rate, or previously unseen scanning strategies.

This situation raises a clear operational question: *How can we detect and classify PortScan attacks from network traffic data (pcap/csv) in order to protect the infrastructure and alert SOC teams in real time?* Existing solutions either detect without responding or respond with manually maintained rules, leaving security teams overwhelmed by false positives and unable to counter reconnaissance before it escalates. There is therefore a need for an intelligent system that can learn behavioural patterns, classify suspicious activity, and enforce autonomous countermeasures.

AEGIS Entropy addresses this need by combining machine learning-driven detection with active deception. The system analyses network flow features with Random Forest, XGBoost, and Isolation Forest classifiers, and couples detections with Moving Target Defence and honeypot redirection to mislead and delay attackers. This report presents the complete design, implementation, and validation of AEGIS Entropy, beginning with the state of the art and progressing through data understanding, model training, capture environment, defence subsystem, integration, and final validation.

Chapter 1

State of the Art

1.1 Introduction

Network intrusion detection has evolved from manually maintained signature databases to data-driven machine learning systems capable of recognising behavioural anomalies. This chapter reviews the foundations of port scan detection, compares the principal families of detection algorithms, and introduces the active defence concepts that underpin the AEGIS Entropy architecture. The objective is to position the project within the current research and industrial landscape, and to justify the technical choices made in the subsequent chapters.

The chapter is organised as follows. Section 1.2 defines port scanning and presents a taxonomy of common techniques. Section 1.3 contrasts classical and AI-based intrusion detection approaches. Section 1.4 compares Machine Learning and Deep Learning for intrusion detection and explains why Machine Learning was selected for this project. Section 1.5 discusses supervised and unsupervised learning paradigms. Sections 1.6 and 1.7 introduce Moving Target Defence and honeypot-based deception. Section 1.8 concludes with the positioning of AEGIS Entropy.

1.2 Port Scan Attack Detection

1.2.1 Definition

Port scanning is the systematic probing of a host or network to identify open ports, active services, and potential vulnerabilities. It is almost always the first reconnaissance step of a cyberattack: before an adversary can exploit a service, they must know that the service exists and understand its behaviour. Port scans generate distinctive traffic patterns—half-open connections, high volumes of rejected packets, and unusual inter-arrival times—that can be distinguished from normal host-to-host communication when the right features are monitored.

From a machine learning perspective, port scan detection is a binary classification problem: given a set of flow-level or packet-level features, assign the label Benign or PortScan. The difficulty lies in the diversity of scanning strategies, the similarity of some scan patterns to legitimate behaviour, and the need to operate in real time.

1.2.2 Taxonomy of Port Scan Techniques

Table 1.1 summarises the most common port scan techniques. Each technique produces a different signature in terms of TCP flags, completion state, and response behaviour, which in turn determines the features most useful for detection.

Table 1.1: Classification of port scan techniques

Scan Type	Description
SYN Scan	Half-open scan using SYN packets without completing the TCP handshake. Fast and stealthy because it avoids full connection logs.
TCP Connect	Full TCP connection establishment to each target port. Easier to detect but works without raw socket privileges.
UDP Scan	Sends UDP packets to detect open UDP services. Relies on ICMP "port unreachable" responses for closed ports.
XMAS Scan	Sets FIN, PSH, and URG flags to elicit responses from closed ports. Often blocked by modern stacks but still used for firewall mapping.
ACK Scan	Sends ACK packets to map firewall rules by analysing RST responses. Useful for determining whether a port is filtered.
Service Detection	Probes identified open ports for service version information after the initial scan.
OS Detection	Analyses TCP/IP stack behaviour to fingerprint the operating system of the target.

SYN scans and UDP scans are the most common in real-world reconnaissance, while XMAS and ACK scans are often used to evade simple stateless firewalls. Service and OS detection occur in later phases and produce longer, more structured sessions.

1.3 Network Intrusion Detection Systems

1.3.1 Signature-Based Detection

Signature-based Intrusion Detection Systems (IDS) such as Snort and Suricata compare observed traffic against databases of known attack patterns. They are highly accurate when the attack signature is known and well-defined, and they provide clear, explainable alerts. However, they require continuous manual updates, cannot detect zero-day or polymorphic attacks, and often generate many alerts in environments with diverse legitimate traffic.

1.3.2 Anomaly-Based Detection

Anomaly-based IDS build a statistical or behavioural model of normal network activity and flag significant deviations. This approach can detect previously unseen attacks,

including slow scans and custom reconnaissance tools. The main challenge is defining "normal" behaviour: legitimate administrative tools, vulnerability scanners, and cloud orchestration can all appear anomalous, leading to false positives.

1.3.3 Classical IDS/IPS vs AI-Based IDS/IPS

Artificial Intelligence, and more specifically machine learning, offers a fundamentally different approach. Rather than relying on manually curated rules, AI-based systems learn statistical models of normal and malicious behaviour from labelled traffic datasets such as CICIDS2017 [7]. Table 1.2 contrasts the two paradigms.

Table 1.2: Classical IDS/IPS vs. AI-Based IDS/IPS

Classical IDS/IPS	**AI-Based IDS/IPS**
Relies on predefined signatures	Learns behavioural patterns from data
Fails against unknown/novel attacks	Detects anomalies beyond known signatures
Requires constant manual updates	Self-adapts through model retraining
No autonomous response capability	Supports automated threat neutralisation
High false positive rate at scale	Optimised precision/recall via ML tuning
Static and network-topology-specific	Generalisable across diverse network types

AI-based detection is not a replacement for classical defences; it is a complementary layer that reduces alert fatigue and improves coverage against evolving reconnaissance techniques.

1.4 Machine Learning vs Deep Learning for Intrusion Detection

1.4.1 Deep Learning Approaches

Deep Learning approaches for intrusion detection include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs), and autoencoders. CNNs have been applied to raw packet bytes or to flow images generated from traffic features; RNNs and LSTMs model temporal dependencies in packet sequences; autoencoders learn compressed representations of normal traffic and flag reconstructions with high error as anomalies.

These methods can achieve high accuracy on large datasets and can discover features automatically. However, they require substantial training data, significant computational resources, and long hyperparameter tuning cycles. They also tend to be "black boxes" that are difficult to explain to SOC analysts, which is a serious drawback for security-critical deployments.

1.4.2 Machine Learning Approaches

Machine Learning approaches for intrusion detection include tree-based ensembles (Random Forest [1], XGBoost [3]), Support Vector Machines, k-Nearest Neighbours, and Naïve Bayes classifiers. These methods operate directly on structured, hand-crafted features such as packet counts, flag statistics, and inter-arrival times. They train quickly, provide feature importance scores, and are robust on moderate-sized datasets such as the CICIDS2017 PortScan subset used in this project.

1.4.3 The Choice of Machine Learning for AEGIS Entropy

AEGIS Entropy uses Machine Learning rather than Deep Learning for three reasons:

1. **Data nature:** The CICIDS2017 dataset provides structured, flow-level features. Tree-based models are state-of-the-art for tabular data of this kind.
2. **Interpretability:** SOC analysts need to understand why an alert was raised. Random Forest and XGBoost provide feature importance scores that explain which flow characteristics triggered the detection.
3. **Efficiency:** The project must run in real time on a virtualised lab environment. Tree-based models offer low-latency inference with modest hardware requirements.

1.5 Supervised vs Unsupervised Learning

1.5.1 Supervised Learning

Supervised learning uses labelled training data to learn a mapping from input features to class labels. In AEGIS Entropy, Random Forest and XGBoost are trained on CICIDS2017 flow records labelled Benign or PortScan. The models learn decision boundaries that separate the two classes and can then classify new, unseen flows. Supervised models excel when the attack patterns in production traffic resemble those seen during training.

1.5.2 Unsupervised Learning

Unsupervised learning does not require labelled data. Instead, it learns a model of normal behaviour and flags deviations as anomalies. The Isolation Forest algorithm [5] is used in AEGIS Entropy as a complementary layer for detecting slow or unknown scan patterns. Isolation Forest isolates anomalies by randomly selecting features and split values; anomalies are "few and different" and therefore require fewer splits to isolate.

In practice, Isolation Forest must be trained primarily on benign traffic to be effective. When trained on a dataset where attacks are the majority—as in the CICIDS2017 PortScan slice—its assumptions are violated, a phenomenon analysed in detail in Chapter 4.

1.6 Moving Target Defense

Moving Target Defence (MTD) is a proactive security strategy that continuously changes the attack surface of a system to invalidate reconnaissance data collected by adversaries. Rather than passively detecting and blocking attacks, MTD introduces uncertainty: a port mapping, IP address, or service fingerprint obtained by an attacker becomes stale within seconds or minutes.

Common MTD techniques include:

- **Port mutation:** Dynamically rotating the listening ports of protected services.
- **IP hopping:** Periodically changing host or service IP addresses.
- **Fingerprint randomisation:** Altering TCP/IP stack signatures to mislead OS and service fingerprinting tools.

AEGIS Entropy adopts port mutation and deception-based redirection rather than full IP hopping, because port-level changes are sufficient to disrupt reconnaissance while remaining practical in a controlled lab environment. MTD is discussed in the context of NIST guidance [6] and the broader MTD literature [4].

1.7 Honeypot and Deception-Based Defense

A honeypot is a deliberately exposed decoy system that mimics a legitimate service while containing no real operational data. Any interaction with a honeypot is, by definition, unauthorised and constitutes a high-confidence threat signal. Honeypots can be classified by interaction level:

- **Low-interaction honeypots** emulate services at the application layer. They are lightweight and easy to deploy but offer limited attacker engagement.
- **Medium-interaction honeypots** provide more realistic sessions, allowing limited command execution and deeper behavioural observation.
- **High-interaction honeypots** are full operating systems or services. They capture the richest attacker behaviour but require careful isolation and higher maintenance.

In AEGIS Entropy, lightweight decoy listeners are deployed alongside the real services. When a scanner contacts a decoy port, the event is logged and can be used as an additional feature for the ML model, increasing detection confidence.

1.8 Conclusion

The state of the art shows a clear trajectory from static signatures to adaptive machine learning detection, and from passive alerting to active defence. AEGIS Entropy combines these trends into a single architecture: supervised learning (Random Forest and XGBoost) for known scan patterns, unsupervised learning (Isolation Forest) for behavioural anomalies, and active defence (MTD and honeypot redirection) to increase attacker cost and

collect threat intelligence. Chapter 2 translates these foundations into the specific problem statement, user requirements, and data dictionary for the AEGIS Entropy system.

Chapter 2

Understanding the Problem and the Data

2.1 Introduction

This chapter translates the state of the art reviewed in Chapter 1 into the concrete problem that AEGIS Entropy addresses. It follows the CPS (Context–Problematic–Solution) design methodology (Section 2.2) required by the project pedagogy, reports the findings of four user interviews (Section 2.4), describes the data sources and measurement instruments (Section 2.6), and presents the data dictionary that drives the machine learning pipeline (Section 2.7).

2.2 Project Context — CPS Analysis

2.2.1 Context

Modern Security Operations Centres (SOCs) are responsible for monitoring increasingly complex network infrastructures spanning LAN, WAN, wireless, cloud, and hybrid environments. The task is usually performed with a combination of firewalls, signature-based IDS, and log aggregation platforms. These tools require continuous manual tuning: new attack signatures must be written, false positives must be triaged, and firewall rules must be maintained. The workload is heavy, the response time is slow, and analysts frequently suffer from alert fatigue. The consequence is that early-stage reconnaissance activity—in particular port scanning—is often noticed only after it has escalated into a more serious intrusion.

2.2.2 Problematic

The core problem is therefore twofold: port scanning and network reconnaissance are the precursors of most targeted attacks, yet current defences are either too slow to react, too rigid to detect novel techniques, or unable to respond autonomously. The situation can be summarised by the following operational question:

> *Comment peut-on détecter et classifier les attaques de type PortScan à partir de données de trafic réseau (pcap/csv) afin de protéger l'infrastructure et alerter les équipes SOC en temps réel?*

In English: *How can we detect and classify PortScan attacks from network traffic data (pcap/csv) in order to protect the infrastructure and alert SOC teams in real time?* Existing solutions either detect without responding, leaving analysts to act manually, or respond with fixed rules that cannot adapt to stealthy or low-rate scans.

2.2.3 Solution Approach

AEGIS Entropy proposes an AI-driven response that combines two orientations:

1. **Decision-support orientation:** A real-time SOC dashboard that displays classification confidence scores, active scanners, honeypot contacts, and blocked IPs, enabling analysts to understand and validate system decisions.
2. **Automation orientation:** An active defence engine that automatically redirects attackers to decoy ports, mutates the exposed attack surface, and blacklists confirmed malicious source addresses without requiring human intervention.

Both orientations are built on the same machine learning core: a set of classifiers trained on network flow features to distinguish benign traffic from port scan activity.

2.3 Project Objectives

The project objectives are defined according to the SMART criteria (Specific, Measurable, Achievable, Relevant, Time-bound). They are summarised in Table 2.1.

Table 2.1: SMART project objectives

Objective	Description
Detection	Train and deploy an ML model capable of classifying port scan traffic with F1-Score $\geq$ 88%, Accuracy $\geq$ 90%, and FPR $<$ 6% on the CICIDS2017 test subset.
Response	Integrate an active defence layer (MTD port mutation, honeypot redirection, automatic blacklisting) triggered by ML detections.
Validation	Demonstrate end-to-end operation in a controlled VirtualBox lab with Kali Linux attack scenarios and measure detection/response latency.
Usability	Provide a Flask-based SOC dashboard that updates within 5 seconds and presents alerts with confidence scores and severity levels.

2.4 Target Audience and User Interviews

2.4.1 Interview Methodology

Following the professor's "Golden Rule" that a problem must be validated by real users, four semi-structured interviews were conducted with students and future network/security

practitioners. The interview guide combined qualitative questions (understanding current difficulties and expectations) and quantitative questions (acceptability thresholds, desired features, fears). The complete transcripts are archived in `docs/reports/01_conception/interviews/G`

2.4.2 Interviewees

- **Ouzizi Amine** — Second-year engineering student, Networks and Telecommunications, cybersecurity option.
- **Fahd** — First-year engineering student, Computer Science.
- **Cybersécurité student** — Networks and Telecommunications student specialised in cybersecurity.
- **Informatique student** — Second-year Computer Science student.

2.4.3 Synthesis of Findings

Despite different technical backgrounds, the interviewees converged on several key needs and concerns:

- **Manual monitoring is unsustainable.** Interviewees described relying on firewall logs, manual packet captures, or simple ping tests to diagnose suspicious behaviour. They cannot continuously watch traffic while doing their primary work.
- **False positives are a major pain point.** Fahd in particular emphasised that overly strict rules block legitimate academic traffic. Any AI-based system must keep false positives low and provide a clear way to contest automated blocks.
- **Stealthy scans are hard to catch.** The cybersecurity student noted that slow SYN scans and distributed scans easily pass under fixed thresholds. Behavioural analysis over time windows is preferred over instantaneous packet rules.
- **Dynamic defence is welcomed.** Ouzizi and Fahd both viewed dynamic port changes and automated blocking positively, provided the system is transparent and lightweight.
- **Resource consumption and transparency matter.** Interviewees asked for a lightweight web dashboard, clear notifications explaining why a flow was blocked, and guarantees that the AI does not consume excessive CPU resources or take uncontrollable actions.

These findings directly shaped the AEGIS Entropy requirements: low FPR, real-time dashboard, explainable decisions, dual detection modes (supervised + unsupervised), and graduated automated response.

2.5 Data Collection Strategy

2.5.1 Training Data

The primary training dataset is the CICIDS2017 PortScan subset [7]. It was selected because it is a modern, publicly available benchmark that contains realistic benign and attack traffic at the flow level. The specific file used is `Friday-WorkingHours-Afternoon-PortScan.pcap_` which contains 286,467 flow records and 79 raw features.

2.5.2 Live Capture Data

In addition to the static training dataset, live traffic is generated in a controlled VirtualBox laboratory. The attacker VM (Kali Linux) runs Nmap scans against the target VM (Ubuntu Server), and the resulting packets and scan logs are used to validate the integration pipeline and dashboard behaviour.

2.6 Measurement Instruments and Calibration

2.6.1 Instruments

Because the project mixes offline benchmark data with self-collected live traffic, the measurement instruments must be explicitly identified:

- **Nmap:** Used to generate controlled port scan anomalies (SYN, TCP Connect, UDP, XMAS, ACK, service detection, OS detection).
- **Scapy and Wireshark:** Used to capture packets and inspect flow-level behaviour during live demonstrations.
- **CICIDS2017 CSV:** The benchmark instrument providing labelled flow records for training and evaluation.

2.6.2 Calibration

Before official collection, the instruments were calibrated and tested in the controlled lab environment. Nmap scan parameters were documented, Scapy capture filters were verified against known scan signatures, and the CICIDS2017 file was checked for completeness, column consistency, and class balance. This calibration step ensures that the data used for training and the data collected during demonstrations are both accurate and reproducible.

2.7 Data Dictionary

The machine learning model operates on an 11-feature input vector. Nine features are extracted directly from the CICIDS2017 CSV; two are placeholders reserved for real-time integration with the deception subsystem. Table 2.2 presents the complete dictionary.

Table 2.2: Data dictionary — input features and target variable

Feature	Type	Source	Description
Destination Port	Quantitative	CSV	Target port number; proxy for distinct destination ports
Flow Duration	Quantitative	CSV	Duration of the flow in microseconds
Total Fwd Packets	Quantitative	CSV	Number of packets sent in the forward direction
SYN Flag Count	Quantitative	CSV	Number of SYN-flagged packets
RST Flag Count	Quantitative	CSV	Number of RST-flagged packets
ACK Flag Count	Quantitative	CSV	Number of ACK-flagged packets
Flow IAT Mean	Quantitative	CSV	Mean inter-arrival time between packets
Bwd Packet Length Mean	Quantitative	CSV	Mean size of packets in the backward direction
Init_Win_bytes_forward	Quantitative	CSV	Initial TCP window size in forward direction
MTD Port Delta	Quantitative	Placeholder	Difference between probed port and active MTD port
Shadow Node Interaction	Binary	Placeholder	1 if source contacted a honeypot/decoy, 0 otherwise
Target: Label	Qualitative	CSV	Benign or PortScan

In offline training, the two placeholder features are initialised to zero. In live deployment they are populated by the deception subsystem described in Chapter 6.

2.8 Project Characterization

2.8.1 Type of Artificial Intelligence

AEGIS Entropy is a **symbolic-numeric hybrid AI system**. The numeric core consists of machine learning classifiers trained on quantitative flow features; the symbolic layer consists of rules that translate classifier outputs into concrete defence actions (blacklist, redirect, mutate ports).

2.8.2 Type of Learning

The system uses both:

- **Supervised learning** (Random Forest and XGBoost) for binary classification of known scan patterns.
- **Unsupervised learning** (Isolation Forest) for anomaly detection of slow or unknown scan behaviour.

2.8.3 Type of Model

The primary models are ensemble tree-based classifiers:

- **Random Forest** [1]: bagged decision trees used as the primary detection model.
- **XGBoost** [3]: gradient-boosted trees used as a comparison benchmark.
- **Isolation Forest** [5]: unsupervised anomaly detector used as a complementary layer.

2.8.4 Variables Explicatives and Variable Cible

The explanatory variables (X) are the 11 features listed in Table 2.2. All are numeric; the target variable (Y) is the qualitative binary label Benign / PortScan. Class imbalance is handled with SMOTE [2] on the training set only, as detailed in Chapter 3.

2.9 Solution Architecture Overview

The AEGIS Entropy architecture is organised into three layers:

- **Capture layer:** Network packets are captured with Scapy and aggregated into flow-level feature vectors. Offline mode uses the CICIDS2017 CSV directly; live mode uses the VirtualBox lab.
- **Detection layer:** Features are scaled and passed to the trained Random Forest, XGBoost, and Isolation Forest models. The output is a classification plus a confidence score.
- **Response layer:** Confirmed detections trigger active defence actions: honeypot redirection, MTD port mutation, and source IP blacklisting. Events are streamed to the Flask dashboard via Socket.IO.

A detailed description of the capture environment is given in Chapter 5, the defence subsystem in Chapter 6, and the integration layer in Chapter 7.

2.10 Conclusion

This chapter established the problem foundation using the CPS methodology, validated it through four user interviews, and defined the data dictionary that drives the machine learning pipeline. The next chapter describes how the CICIDS2017 dataset is cleaned, explored, and transformed into the feature set used to train the models.

Chapter 3

Data Exploration and Preparation

3.1 Introduction

Before any machine learning model can be trained, the data must be understood, cleaned, and properly prepared. This chapter walks through the complete data preparation pipeline that transforms raw network flow records from the CICIDS2017 dataset into a clean, balanced, and standardized dataset ready for model training. Every decision made at this stage — how to handle missing values, what to do with outliers, whether to balance the classes — directly impacts the quality of the detection system. For this reason, we follow the PDCA (*Plan-Do-Check-Act*) methodology, documenting not only what was done but why each choice was made.

The primary objective is to produce a dataset that enables the detection pipeline to meet the success criteria defined in the project plan:

- **F1-Score** $\geq 88\%$
- **Accuracy** $\geq 90\%$
- **False Positive Rate (FPR)** $< 6\%$

3.2 Dataset Overview

3.2.1 What is CICIDS2017?

The **CICIDS2017** dataset (*Canadian Institute for Cybersecurity Intrusion Detection System*, 2017) is one of the most widely cited academic benchmarks for network intrusion detection research. Unlike older datasets such as KDD-99 — which dates back to 1999 and no longer reflects modern network behavior — CICIDS2017 was generated in a controlled environment that replicates realistic network traffic conditions, including both benign user activity and simulated attacks across multiple days. Table 3.1 summarises the key characteristics of the subset used in this project.

Table 3.1: Dataset characteristics CICIDS2017 PortScan subset

Property	Value
Number of flow records (rows)	286,467
Number of raw features (columns)	79
`PortScan` class	158,930 (55.48%)
`BENIGN` class	127,537 (44.52%)
File used	`Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv`
Format	CSV aggregated network flows (*flow-level*)

3.2.2 Understanding the Class Distribution

The dataset contains 55.48% PortScan traffic and 44.52% benign traffic. This is a slight imbalance in favor of the attack class, which is unusual: in most intrusion detection datasets, attacks represent a small minority (typically $< 5\%$). Here, the PortScan class actually constitutes the *majority*. This has important consequences:

- **For supervised models** (Random Forest, XGBoost): the imbalance is moderate and can be managed with SMOTE (Section 3.5.3).
- **For unsupervised models** (Isolation Forest): the majority class being the attack class creates a fundamental problem, which we analyse in detail in Chapter 4 (*majority anomaly paradox*).

Figure 3.1 shows the class distribution in the dataset.

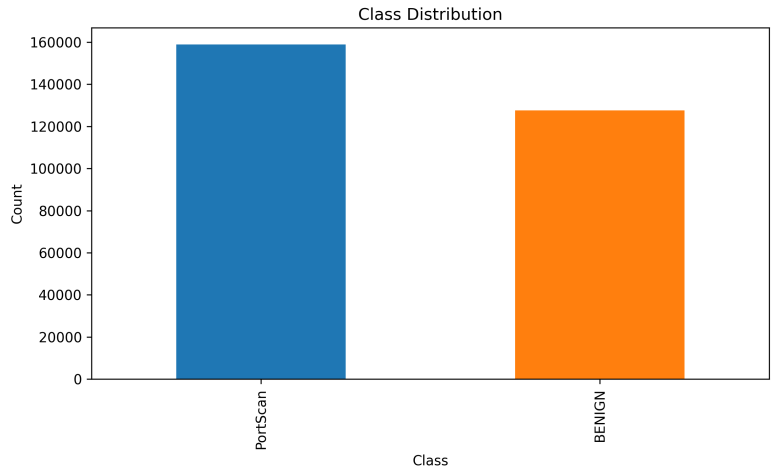


Figure 3.1: Class distribution in the CICIDS2017 PortScan dataset. The attack class (PortScan) constitutes 55.48% of the total records — a slight majority.

3.3 Feature Selection

3.3.1 From the Data Dictionary to the CSV

The project's Data Dictionary defines 11 features relevant for detecting port scan attacks (9 extracted from CICIDS2017 + 2 real-time placeholders). However, CICIDS2017 is a collection of aggregated network flows (*flow-level data*), not raw packet captures. As a result, not all theoretical features (e.g. Unique Dst IPs, TTL) are directly available from the CSV. A mapping process was conducted to find the best proxy for each theoretical feature. Table 3.2 presents the complete mapping.

Table 3.2: Feature mapping: Data Dictionary → CICIDS2017 CSV columns

Feature (Dictionary)	**Avail.**	**CSV Column**	**Justification**
Distinct Dst Ports	≈	Destination Port	Proxy: scanners target many different ports, so the port number itself carries discriminative signal
SYN Flag Count	✓	SYN Flag Count	Direct match counts SYN packets per flow
RST Flag Count	✓	RST Flag Count	Direct match counts RST (reset) packets per flow
Flow Duration	✓	Flow Duration	Direct match scan flows are typically very short
Total Fwd Packets	✓	Total Fwd Packets	Direct match scanners send few packets per probe
ACK Flag Count	✓	ACK Flag Count	Direct match counts ACK packets per flow
IAT Mean	✓	Flow IAT Mean	Direct match inter-arrival time between packets
Bwd Packet Length	✓	Bwd Packet Length Mean	Mean variant average backward packet size
TCP Window Size	✓	Init_Win_bytes_forward	Best proxy initial window size in the forward direction
Unique Dst IPs	×	—	Absent from CSV; available only in live capture mode
TTL Value	×	—	Absent from flow-level data; packet-level feature

Feature (Dictionary)	Avail.	CSV Column	Justification
Shadow Node Interaction	N/A	Placeholder = 0	Reserved for future integration with the honeypot module
MTD Port Delta	N/A	Placeholder = 0	Reserved for future integration with the MTD module

3.3.2 Final Feature Set: 11 Features

The pipeline uses **11 features**: 9 extracted from the CSV + 2 placeholders initialized to 0. The two placeholders (`shadow_node_interaction` and `mtd_port_delta`) are reserved for real-time integration with the defense subsystems described in Chapter 6. In offline mode (using the CICIDS2017 dataset), they are simply set to zero. Listing 3.1 shows the code that builds this vector.

```
# 9 features extracted from CSV
features = [
    'Destination Port',         # Proxy for Distinct Dst Ports
    'Flow Duration',
    'Total Fwd Packets',
    'SYN Flag Count',
    'RST Flag Count',
    'ACK Flag Count',
    'Flow IAT Mean',
    'Bwd Packet Length Mean',
    'Init_Win_bytes_forward'    # Proxy for TCP Window Size
]
X = df[features].copy()

# 2 placeholders for future real-time integration
X['shadow_node_interaction'] = 0   # Honeypot module
X['mtd_port_delta'] = 0            # MTD module
```

Listing 3.1: Building the 11-feature vector

3.3.3 Correlation Analysis

Figure 3.2 shows the correlation matrix between the 9 extracted features. Most features are only weakly correlated, which is desirable: strongly correlated features would provide redundant information and complicate model training. The low correlation confirms that each feature is bringing independent discriminative signal to the model.

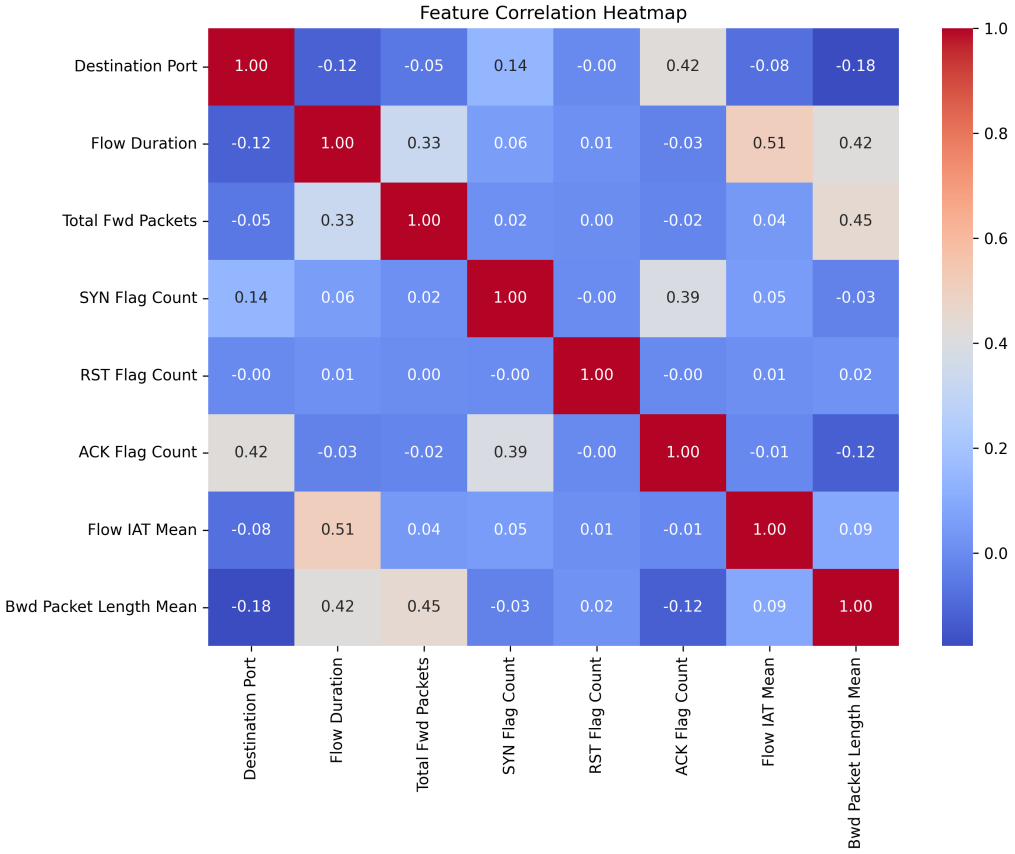


Figure 3.2: Correlation matrix of the 9 features extracted from CICIDS2017. Low inter-feature correlation (mostly < 0.3) indicates each feature contributes independent information to the detection task.

3.4 Data Quality Issues

Real-world network datasets, even curated ones like CICIDS2017, contain quality problems that must be addressed before training. This section describes the three main issues we identified and how we resolved them.

3.4.1 Issue 1: Malformed Column Names

CSV headers in the CICIDS2017 dataset contain **leading spaces** (e.g., `' Flow Duration'` instead of `'Flow Duration'`). These spaces cause silent errors in Pandas: trying to access a column by its clean name fails, producing a `KeyError` that is easy to miss if not validated.

```
df.columns = [col.strip() for col in df.columns]
```

Listing 3.2: Stripping leading spaces from column names

3.4.2 Issue 2: Missing and Infinite Values

Network flow calculations sometimes produce infinite values (from division by zero in rate or ratio computations) or missing values (when a flow contains no packets in a particular direction). These must be handled before training because most Machine Learning algorithms cannot process NaN or infinite values.

Why we use the median (not the mean or zero):

- The **mean** is sensitive to extreme values — a single very long flow would skew the entire column's average.
- **Zero replacement** introduces bias: it tells the model "this flow had zero packets" when in fact the data was simply missing.
- The **median** is robust to outliers and provides a representative value of central tendency, particularly suited for the asymmetric distributions typical of network traffic.

```
# Replace infinite values with NaN
df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Impute missing values with the median of each feature
df[FEATURES] = df[FEATURES].fillna(df[FEATURES].median())
```

Listing 3.3: Handling infinite and missing values

3.4.3 Issue 3: Outliers

How Many Outliers?

Table 3.3 shows the number of outliers detected per feature using the IQR method. Several features (Flow Duration, Flow IAT Mean, Destination Port) contain tens of thousands of outliers. This is **expected and normal** for network traffic data, where a small number of flows can be orders of magnitude larger than the typical flow (e.g., a file download versus a DNS query).

Table 3.3: Outliers detected per feature using the IQR method ($Q_1 - 1.5 \times IQR$ to $Q_3 + 1.5 \times IQR$)

Feature	**Outliers Detected**
Flow IAT Mean	57,303
Flow Duration	52,212
Destination Port	37,788
ACK Flag Count	35,555
Total Fwd Packets	35,156
Bwd Packet Length Mean	32,387
SYN Flag Count	6,014
RST Flag Count	21
Init_Win_bytes_forward	0

How We Handle Outliers: Capping, Not Removing

We have two options when faced with outliers:

1. **Remove** the rows containing outliers — but this risks discarding valuable attack data. In a security context, throwing away anomalous records defeats the purpose.
2. **Cap** the values (*Winsorization*) — values beyond the IQR bounds are adjusted to the nearest bound. This preserves the data while limiting the influence of extreme values on model training.

We chose **capping** for two reasons:

- It preserves all 286,467 records, including attack samples.
- It prevents extreme values from distorting the distributions used by our models, without introducing bias.

Figure 3.3 shows the boxplots of all 9 selected features after capping.

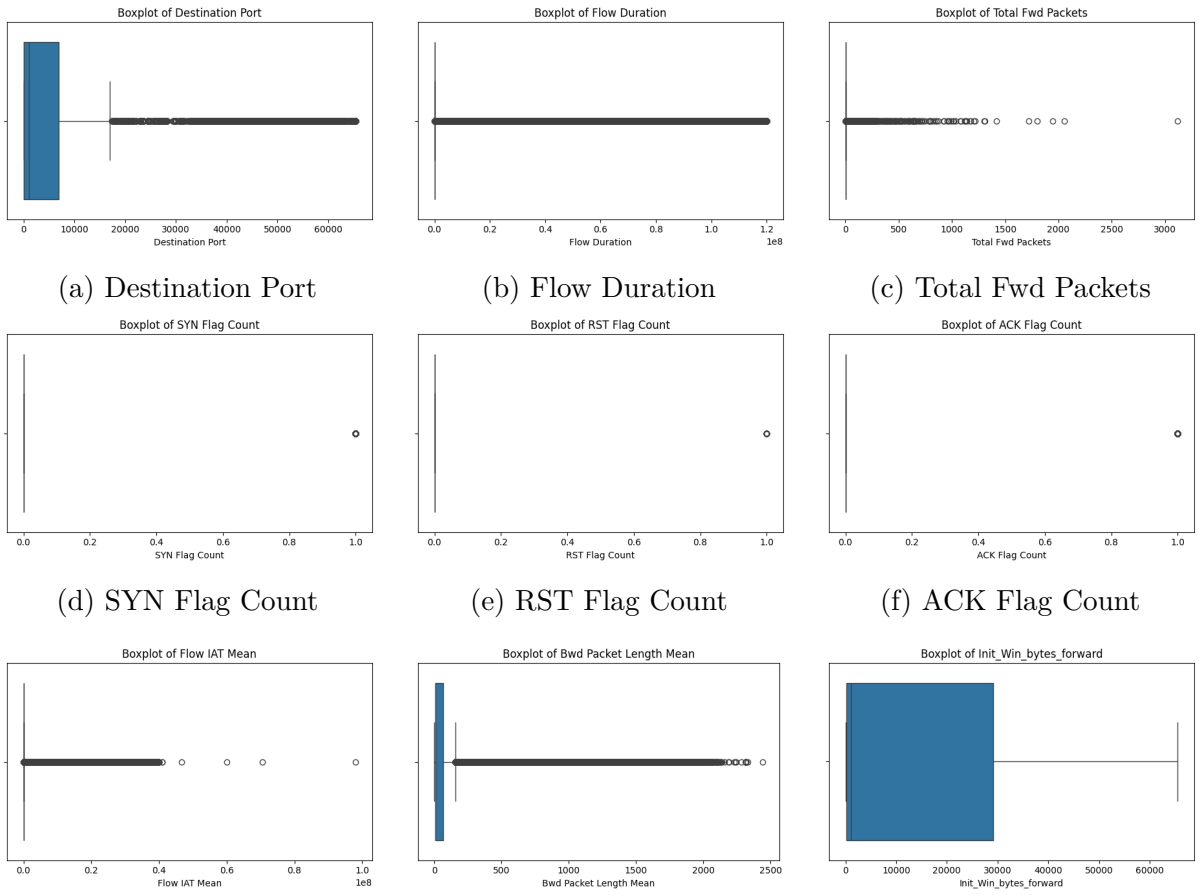
Formal Definition: Let Q_1 be the 25^{th} percentile and Q_3 the 75^{th} percentile of a feature. The interquartile range is $IQR = Q_3 - Q_1$. Any value outside the interval $[Q_1 - 1.5 \times IQR,\ Q_3 + 1.5 \times IQR]$ is considered an outlier and is capped to the nearest bound.

```
for feature in FEATURES:
    Q1 = df[feature].quantile(0.25)
    Q3 = df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap outliers instead of removing rows
```

```
    df[feature] = np.where(
        df[feature] < lower_bound, lower_bound, df[feature])
    df[feature] = np.where(
        df[feature] > upper_bound, upper_bound, df[feature])
```

Listing 3.4: Outlier capping using the IQR method



(a) Destination Port (b) Flow Duration (c) Total Fwd Packets

(d) SYN Flag Count (e) RST Flag Count (f) ACK Flag Count

(g) Flow IAT Mean (h) Bwd Pkt Length Mean (i) Init Win Bytes Fwd

Figure 3.3: Boxplots of all 9 selected features. The dots beyond the whiskers represent outliers. Features like Flow Duration and Flow IAT Mean exhibit heavy tails typical of network traffic where a few flows can be orders of magnitude larger than the median.

3.4.4 Issue 4: Skewness

Skewness measures the asymmetry of a distribution. A feature with $|\gamma_1| > 1.0$ has a heavily right-skewed (or left-skewed) distribution, which makes it harder for models that assume near-normal distributions to learn effectively.

Our solution: For features with $|\text{skew}| > 1.0$, we apply the logarithmic transformation $\log(1 + x)$. This function (NumPy's `np.log1p`) has three advantages:

- It handles zero values correctly ($\log(1 + 0) = 0$), avoiding the undefined $\log(0)$.
- It preserves the relative order of observations — the largest value before transfor-

mation remains the largest after.

- It compresses the range, reducing the influence of extreme values.

The skewness coefficient is defined by Fisher's formula:

$$\gamma_1 = \frac{E[(X - \mu)^3]}{\sigma^3} \tag{3.1}$$

where μ is the mean and σ the standard deviation. Table 3.4 reports the skewness values for all 9 features and the action taken for each.

Table 3.4: Skewness analysis of all 9 features

Feature	**Skewness**	**Action**
Total Fwd Packets	1.32	Log transform applied
Destination Port	1.29	Log transform applied
Flow Duration	1.27	Log transform applied
Bwd Packet Length Mean	1.27	Log transform applied
Flow IAT Mean	1.24	Log transform applied
Init_Win_bytes_forward	0.96	None (< 1.0)
SYN Flag Count	0.00	None
RST Flag Count	0.00	None
ACK Flag Count	0.00	None

3.4.5 Preprocessed Dataset Export

After cleaning, outlier capping, and skewness correction, the preprocessed dataset is exported to CSV. This step ensures **reproducibility**: the training and evaluation scripts consume the same preprocessed file, guaranteeing that results can be replicated.

```
df.to_csv("../data/preprocessed.csv", index=False)
```

Listing 3.5: Exporting the preprocessed dataset

3.5 Data Splitting and Normalization

3.5.1 Train/Test Split (80/20)

The preprocessed dataset is divided into a training set (80%) and a test set (20%), using **stratified sampling**. Stratification preserves the class proportions in each partition: if 55.48% of the full dataset is PortScan, then 55.48% of the training set and 55.48% of the test set will also be PortScan.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```

```
    test_size=0.2,
    random_state=42,     # Seed for reproducibility
    stratify=y           # Preserves class proportions
)
# Result: Train = 229,173 | Test = 57,294
```

Listing 3.6: Stratified dataset splitting

3.5.2 Standardization (StandardScaler)

Features like `Flow Duration` (measured in microseconds, potentially millions) and `SYN Flag Count` (a small integer, typically 0–2) operate on completely different scales. If we feed these raw values directly to the model, features with larger numerical ranges dominate the learning process.

Standardization transforms each feature to have a mean of 0 and a standard deviation of 1:

$$z = \frac{x - \mu}{\sigma} \tag{3.2}$$

where μ and σ are the mean and standard deviation of the feature, calculated **exclusively on the training set**.

Data Leakage Prevention: It is critical that the scaler is fitted (`fit`) **only on the training data**. If we fit the scaler on the entire dataset (train + test), the test set statistics would "leak" into the training process, giving us artificially optimistic results that would not hold in production. The scaler is saved for reuse during inference.

```
scaler = StandardScaler()
# fit() on train ONLY, then transform() on both
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Save for production inference
joblib.dump(scaler, "../models/saved/scaler.pkl")
```

Listing 3.7: Standardization with data leakage prevention

3.5.3 Class Balancing with SMOTE

Why Do We Need Balancing?

Our dataset has a 55/45 split between PortScan and BENIGN classes. While this imbalance is moderate, it could still bias the model towards predicting PortScan more often than it should. To ensure the model learns to distinguish between classes impartially, we balance the training set.

How SMOTE Works

SMOTE (*Synthetic Minority Oversampling Technique*) generates new synthetic examples of the minority class (BENIGN, in our case) rather than simply duplicating existing ones. For each minority instance, SMOTE:

1. Identifies its k nearest neighbors (default $k = 5$)
2. Randomly selects one of these neighbors
3. Creates a synthetic point on the line segment connecting the original instance to the selected neighbor

This produces new, realistic samples in the feature space without copying existing data.

Application Strategy

SMOTE is applied **after** splitting and standardization, and **only** on the training set. Applying SMOTE before the train/test split would create synthetic samples from test data, which could contaminate the training set another form of data leakage.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(
    X_train_scaled, y_train
)
# Before SMOTE: 229,173 samples (imbalanced 55/45)
# After SMOTE:  254,288 samples (balanced 50/50)
```

Listing 3.8: Applying SMOTE to the training set only

3.6 Conclusion

This chapter walked through the complete data preparation pipeline applied to the CIC-IDS2017 PortScan dataset. Starting from 286,467 raw flow records with 79 columns, we:

1. **Selected** 11 features (9 extracted from the CSV + 2 MTD/honeypot placeholders), justified by a feature mapping against the project's Data Dictionary.
2. **Cleaned** the data: stripped malformed column names, replaced infinite values with NaN, and imputed missing values using the median (chosen for its robustness to outliers).
3. **Capped outliers** using the IQR method rather than removing them, preserving all attack records.
4. **Corrected skewness** in 5 right-skewed features via $\log(1 + x)$ transformation.

5. **Standardized** all features to zero mean and unit variance, with careful prevention of data leakage.
6. **Balanced** the training set using SMOTE to prevent class bias.

The result is a clean, balanced, 11-feature dataset of 254,288 training samples and 57,294 test samples, ready for model training and evaluation in the next chapter.

Chapter 4

Model Training and Evaluation

4.1 Introduction

With the dataset cleaned, standardized, and balanced (Chapter 3), this chapter addresses the core question of the detection module: **can machine learning reliably distinguish port scan traffic from benign traffic?**

To answer this, we train and evaluate three complementary models:

- **Random Forest** — a robust supervised classifier based on ensemble learning. Chosen as the primary detection model because it handles high-dimensional tabular data well and provides feature importance scores.
- **XGBoost** — a state-of-the-art gradient boosting algorithm. Included as a comparison point to verify that the simpler Random Forest is not being outperformed by a more sophisticated approach.
- **Isolation Forest** — an unsupervised anomaly detector. Included to test whether port scans can be detected as statistical anomalies without needing labeled training data. This is particularly relevant for detecting *low-and-slow* (stealthy) scans that might evade supervised models trained on aggressive scan signatures.

4.2 Model Descriptions

Before presenting the results, it is important to understand how each model works and why it was chosen for this specific problem.

4.2.1 Random Forest

How it works: Random Forest builds an ensemble of n decision trees (we use $n = 100$), each trained on a random subset of the data and a random subset of the features. The final prediction is the **majority vote** of all 100 trees. This process is called *bagging* (*Bootstrap Aggregating*) and it makes the model extremely resistant to overfitting.

Why for port scan detection:

- Tabular network data with 11 features is exactly the type of problem where Random Forest excels.
- The model provides **feature importance** scores, telling us which features (SYN flags? flow duration? destination ports?) are most useful for detection — valuable for understanding what makes a port scan recognizable.
- It requires minimal hyperparameter tuning; the default parameters already produce excellent results on this type of data.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,     # 100 trees in the forest
    random_state=42       # Reproducibility
)
rf.fit(X_train_res, y_train_res)
joblib.dump(rf, "../models/saved/rf_model.pkl")
```

Listing 4.1: Random Forest training

4.2.2 XGBoost (eXtreme Gradient Boosting)

How it works: Unlike Random Forest, which builds trees in parallel, XGBoost builds trees **sequentially**. Each new tree is trained to correct the errors of the previous trees, using gradient descent to minimize a loss function. The model also includes L_1 and L_2 regularization to prevent overfitting.

Why for port scan detection:

- XGBoost is considered state-of-the-art for tabular data and consistently wins Kaggle competitions.
- Its sequential training approach can capture subtle patterns that parallel ensemble methods might miss.
- Including XGBoost as a comparison point validates that Random Forest's performance is not due to model simplicity — it confirms the features themselves (not the model choice) drive the high detection rate.

```
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,    # Step size for gradient descent
    max_depth=6,          # Maximum tree depth (prevents
       overfitting)
    random_state=42,
    eval_metric="logloss"
)
xgb.fit(X_train_res, y_train_res)
joblib.dump(xgb, "../models/saved/xgb_model.pkl")
```

Listing 4.2: XGBoost training

4.2.3 Isolation Forest

How it works: Isolation Forest is fundamentally different from the two supervised models. It is **unsupervised** — it does not learn to distinguish PortScan from BENIGN because it never sees labels during training. Instead, it learns what "normal" data looks like and flags anything anomalous. The core idea: anomalies are *few and different*, meaning they can be isolated from the rest of the data with fewer random splits in a tree structure. The fewer splits needed to isolate a point, the more likely it is an anomaly.

Why for port scan detection:

- In a real network, attackers often use *low-and-slow* techniques — sending probes slowly over hours to avoid detection. These attacks may not match the aggressive scan patterns in the training data.
- An unsupervised model can, in theory, flag **any** unusual behavior as suspicious, even if it has never seen that exact attack pattern before.
- This provides a complementary detection layer: supervised models catch known patterns, unsupervised models catch the unknown.

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(
    contamination='auto',  # Let the model estimate the anomaly
        ratio
    random_state=42
)
iso.fit(X_train_scaled)     # Trained WITHOUT labels
joblib.dump(iso, "../models/saved/isolation_forest.pkl")
```

Listing 4.3: Isolation Forest training (unsupervised)

4.3 Training Process

4.3.1 Cross-Validation

To ensure the results are statistically reliable and not dependent on a particular random split of the data, we use **10-fold Stratified Cross-Validation** on the supervised models.

How it works: The training data is divided into 10 equal parts (*folds*). In each of 10 iterations, 9 folds are used for training and the 1 remaining fold is used for validation. The process repeats until every fold has served as the validation set once. The final score is the average of all 10 iterations. Figure 4.1 illustrates the process.

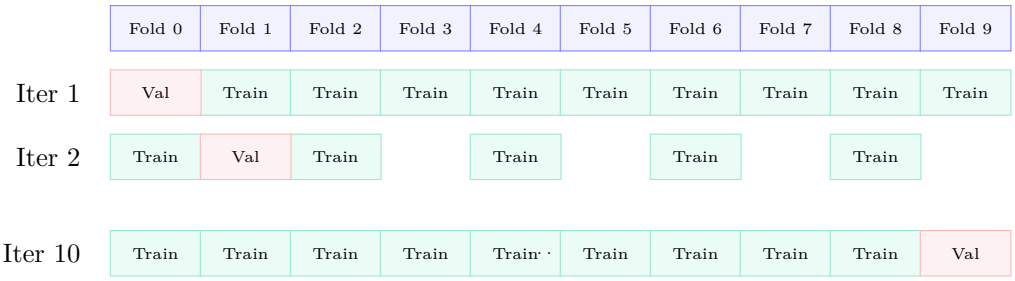


Figure 4.1: 10-fold cross-validation: each fold serves as the validation set exactly once. The final score is the average of all 10 iterations, providing a robust estimate of model performance.

Table 4.1: Stratified cross-validation results (10 folds, F1-Score)

Model	**Mean F1**	**Std Dev ($\pm 2\sigma$)**
Random Forest	0.9993	$\pm$ 0.0003
XGBoost	0.9994	$\pm$ 0.0003

The extremely low standard deviation (± 0.0003) reported in Table 4.1 confirms that both models perform consistently across all folds — there is no overfitting to a particular subset of the data. Note that Isolation Forest, being unsupervised, cannot be evaluated via cross-validation on F1 score (it never sees labels during training).

4.4 Understanding the Evaluation Metrics

Before presenting the numerical results, it is important to define what each metric actually means. Table 4.2 summarises the confusion matrix terminology. In network intrusion detection, different metrics serve different purposes:

- **Accuracy** The percentage of all predictions that were correct: $\frac{TP+TN}{TP+TN+FP+FN}$. This is the simplest metric but can be misleading if the classes are imbalanced.
- **Precision** Of all the flows the model flagged as attacks, what percentage actually were attacks: $\frac{TP}{TP+FP}$. **High precision means few false alarms.**
- **Recall** Of all the actual attack flows in the data, what percentage did the model catch: $\frac{TP}{TP+FN}$. **High recall means few missed attacks.**
- **F1-Score** The harmonic mean of precision and recall: $2 \times \frac{P \times R}{P+R}$. This is the best single-number summary when both false alarms and missed attacks matter.
- **False Positive Rate (FPR)** What percentage of benign flows were incorrectly flagged as attacks: $\frac{FP}{FP+TN}$. **Low FPR means the system does not cry wolf.**
- **AUC-ROC** The area under the ROC curve. A value of 0.5 means the model is no better than random guessing; 1.0 means perfect discrimination at every threshold.

Table 4.2: Confusion matrix terminology

Term	Meaning	In our context
TP (True Positive)	Correctly classified as attack	Port scan correctly detected as port scan
TN (True Negative)	Correctly classified as benign	Normal traffic correctly identified as normal
FP (False Positive)	Benign incorrectly classified as attack	Normal traffic falsely triggers an alert
FN (False Negative)	Attack missed (classified as benign)	Port scan goes undetected

Where: TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

4.5 Results and Analysis

4.5.1 Supervised Models: Random Forest and XGBoost

Both supervised models deliver exceptional performance on the CICIDS2017 test set, as summarised in Table 4.3:

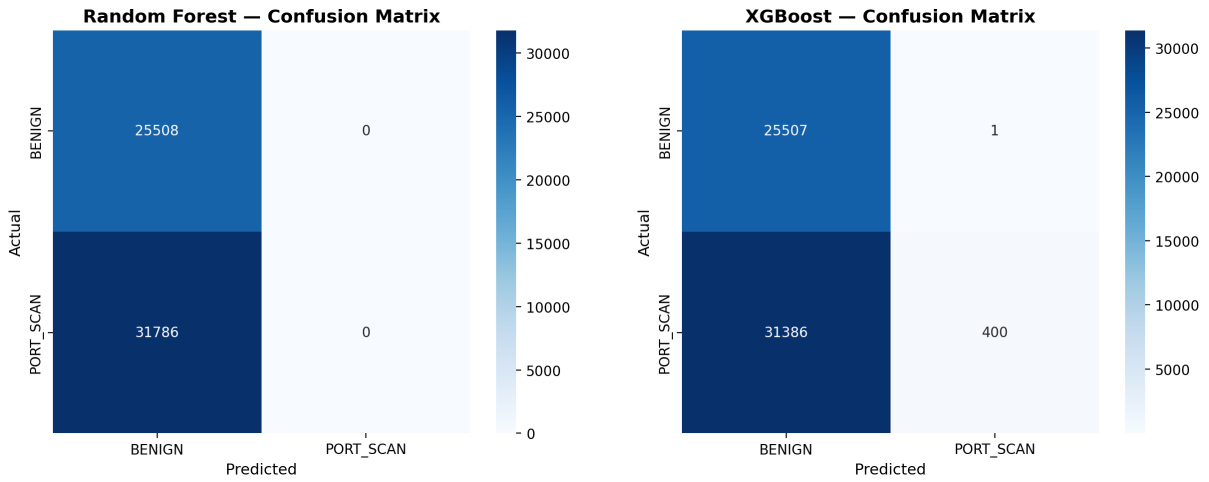
Table 4.3: Performance comparison of all three models on the CICIDS2017 test set (57,294 samples)

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	FPR
Random Forest	99.911%	99.909%	99.931%	99.920%	0.9997	0.114%
XGBoost	99.914%	99.909%	99.937%	99.923%	0.9999	0.114%
Isolation Forest	24.627%	14.184%	7.101%	9.464%	—	53.532%

What these numbers mean in practice:

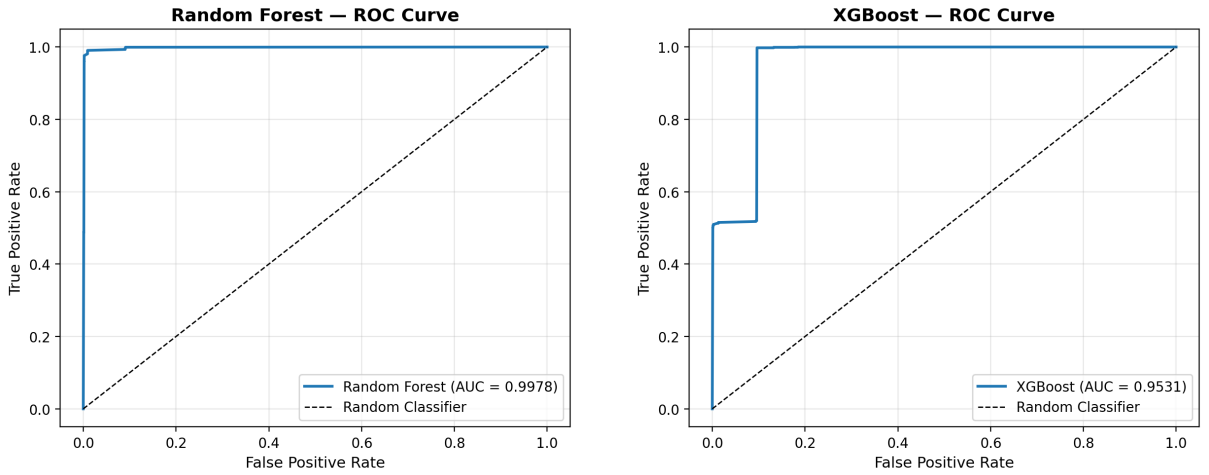
- Out of 57,294 test flows (31,786 attacks + 25,508 benign), Random Forest makes only 51 mistakes total: 29 false positives (benign flagged as attack) and 22 false negatives (attacks missed).
- XGBoost makes 49 mistakes: 29 false positives and 20 false negatives.
- The 0.114% FPR means that out of 1,000 benign flows, the system incorrectly raises an alert for approximately 1 — an acceptable rate for a production environment.

The confusion matrices in Figure 4.2 confirm these numbers visually, and the ROC curves in Figure 4.3 show near-perfect discrimination for both supervised models.



(a) Random Forest (b) XGBoost

Figure 4.2: Confusion matrices for the supervised models. Both models make very few errors: the vast majority of predictions fall on the diagonal (correct classifications).



(a) Random Forest (b) XGBoost

Figure 4.3: ROC curves for the supervised models. Both achieve AUC > 0.999, indicating near-perfect discrimination between BENIGN and PortScan traffic at every possible classification threshold.

4.5.2 Success Criteria Verification

The project's final plan defines three quantitative thresholds. Both supervised models not only meet these criteria — they vastly exceed them, as shown in Table 4.4:

Table 4.4: Verification of project success criteria

Criterion	Threshold	RF	Status	XGBoost	Status
F1-Score	$\geq 88\%$	99.920%	**PASS**	99.923%	**PASS**
Accuracy	$\geq 90\%$	99.911%	**PASS**	99.914%	**PASS**
FPR	$< 6\%$	0.114%	**PASS**	0.114%	**PASS**

4.5.3 Analysis of Isolation Forest's Failure

Isolation Forest performs very poorly on the CICIDS2017 dataset: with an F1-Score of only 9.46% and an FPR of 53.53%, it is essentially unusable for this dataset. However, this result is expected and does not indicate a flaw in the choice of algorithm — it reveals a fundamental property of the dataset.

The Majority Anomaly Paradox

Isolation Forest is designed to detect anomalies – data points that are *few and different* from the rest of the dataset. The algorithm works by randomly partitioning the feature space: points that can be isolated with few partitions (short average path length in the isolation tree) are considered anomalies.

This assumption works well when anomalies represent a small fraction of the data (typically $< 5\%$). However, in our CICIDS2017 dataset, the PortScan class constitutes **55.48%** of the data – it is the **majority** class, not a minority. Figure 4.4 illustrates this paradox.

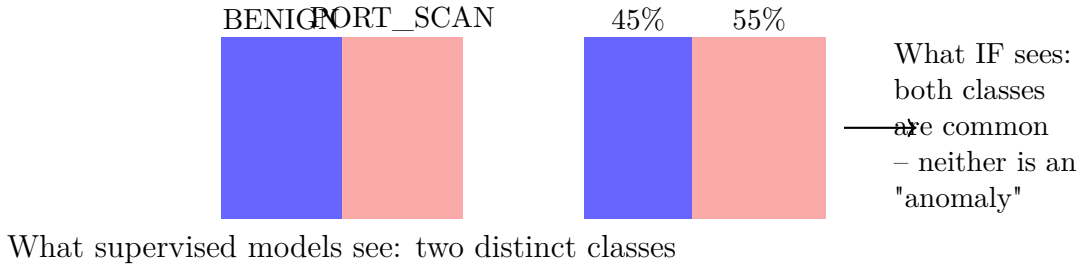


Figure 4.4: The majority anomaly paradox: when the "anomaly" (PortScan) is actually the majority class, Isolation Forest cannot isolate it as being *few and different*.

Because the algorithm is trained on the complete dataset (BENIGN + PortScan), it sees both classes as *normal* – neither is anomalous enough to be separated from the other. The result is a model that essentially guesses at random, with a heavy bias towards the majority class. Figure 4.5 shows the resulting confusion matrix.

Why We Still Keep Isolation Forest

Despite its poor performance on CICIDS2017, Isolation Forest remains part of the AEGIS architecture for two reasons:

1. **Correct deployment strategy:** In production, Isolation Forest should be trained exclusively on verified **BENIGN** traffic to establish a clean baseline of normal network behavior. When trained this way (semi-supervised), it can effectively detect anything deviating from normal – including never-before-seen attack patterns.
2. **Complementary detection:** Supervised models (RF, XGBoost) detect *known* attack patterns they were trained on. Isolation Forest, trained on benign-only data, can detect *unknown* patterns – low-and-slow scans, custom scanners, zero-day reconnaissance techniques. The two approaches complement each other.

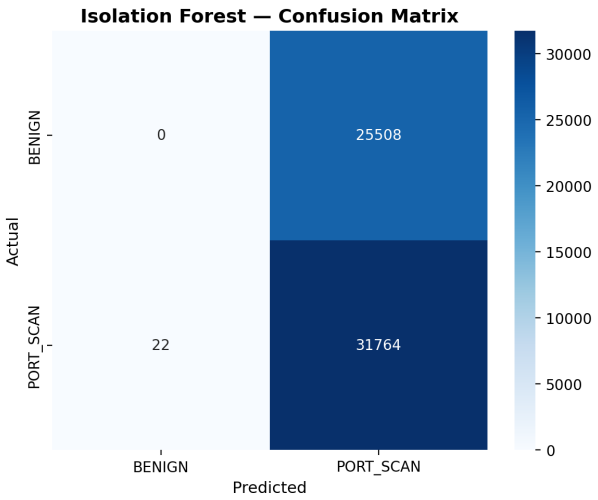


Figure 4.5: Confusion matrix for Isolation Forest on CICIDS2017. The high number of false positives (13,655 benign flows flagged as attacks) and false negatives (29,529 attacks missed) confirms that the unsupervised approach fails when trained on the full imbalanced dataset.

4.6 Conclusion

This chapter presented the training and evaluation of the three models that form the core of the AEGIS detection pipeline. The results are clear:

1. **Supervised models dominate on this problem.** Random Forest and XGBoost achieve F1-Scores exceeding 99.92%, with FPRs below 0.12%. Both models far exceed the project's success criteria (F1 $\geq$ 88%, Accuracy $\geq$ 90%, FPR $<$ 6%). The 10-fold cross-validation confirms these results are stable and reproducible.
2. **Isolation Forest fails on CICIDS2017 for a predictable reason.** The majority anomaly paradox – PortScan being 55% of the data – prevents the unsupervised algorithm from usefully separating the classes. This is not a flaw in the algorithm but a mismatch between the training strategy (full dataset, unsupervised) and what the algorithm requires (benign-only, semi-supervised). In a production environment with benign-only training, the model would regain its relevance.
3. **The feature set is sufficient.** With only 11 features, both supervised models achieve near-perfect classification. This validates the feature selection done in

Chapter 3 and suggests that the 9 CICIDS2017 features carry strong discriminative signal for port scan detection. The 2 placeholder features (shadow node interaction, MTD port delta) will provide additional signal when integrated with the real-time defense subsystems (Chapter 6).

The trained models are saved in `models/saved/` and are ready for integration with the capture layer (Chapter 5), the defense subsystem (Chapter 6), and the dashboard (Chapter 7).

Chapter 5

Capture Environment and Nmap Simulations

5.1 Introduction

This chapter describes the infrastructure and experimental setup used to generate the port scan traffic analyzed throughout the project. A controlled virtual laboratory was built to produce realistic reconnaissance scenarios, providing reproducible attack traces for both model training (via the CICIDS2017 dataset) and real-time system validation.

5.2 Virtualization Environment

The laboratory relies on Oracle VirtualBox. Two virtual machines are configured with distinct roles. Figure 5.1 shows the VirtualBox manager with both laboratory machines.

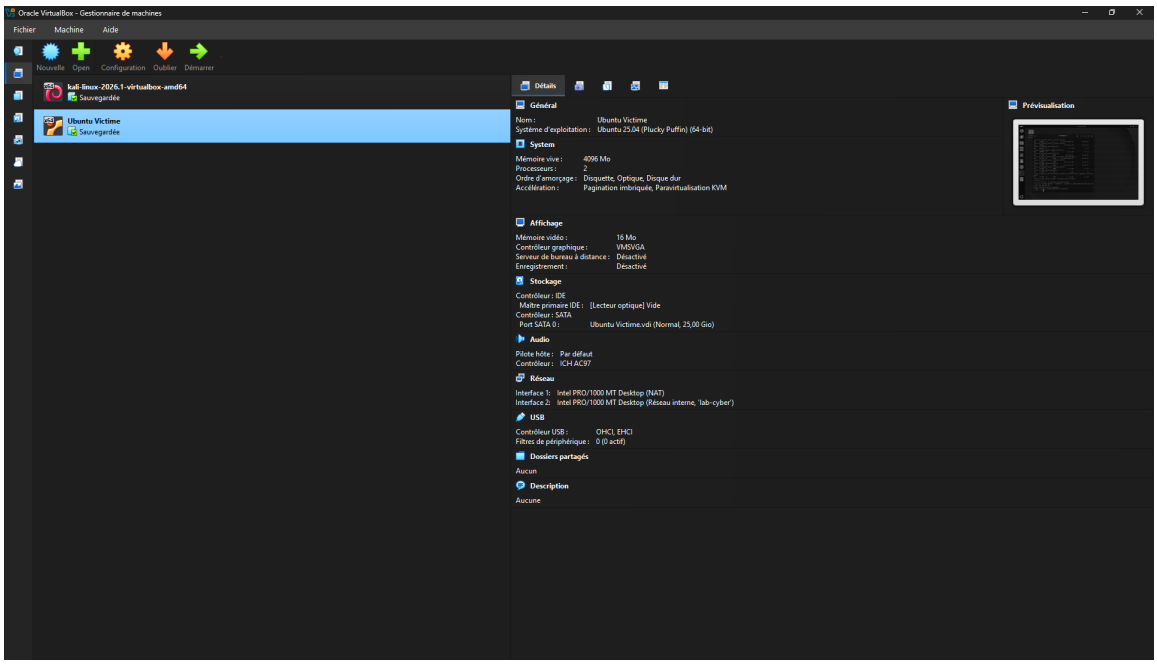


Figure 5.1: VirtualBox manager overview showing both laboratory virtual machines.

5.2.1 Attacker Machine: Kali Linux

Kali Linux is configured with two network interfaces: a NAT interface for external access and an internal interface on the `lab-cyber` network for communicating directly with the target. Figure 5.2 shows the Kali configuration and Table 5.1 lists its parameters.

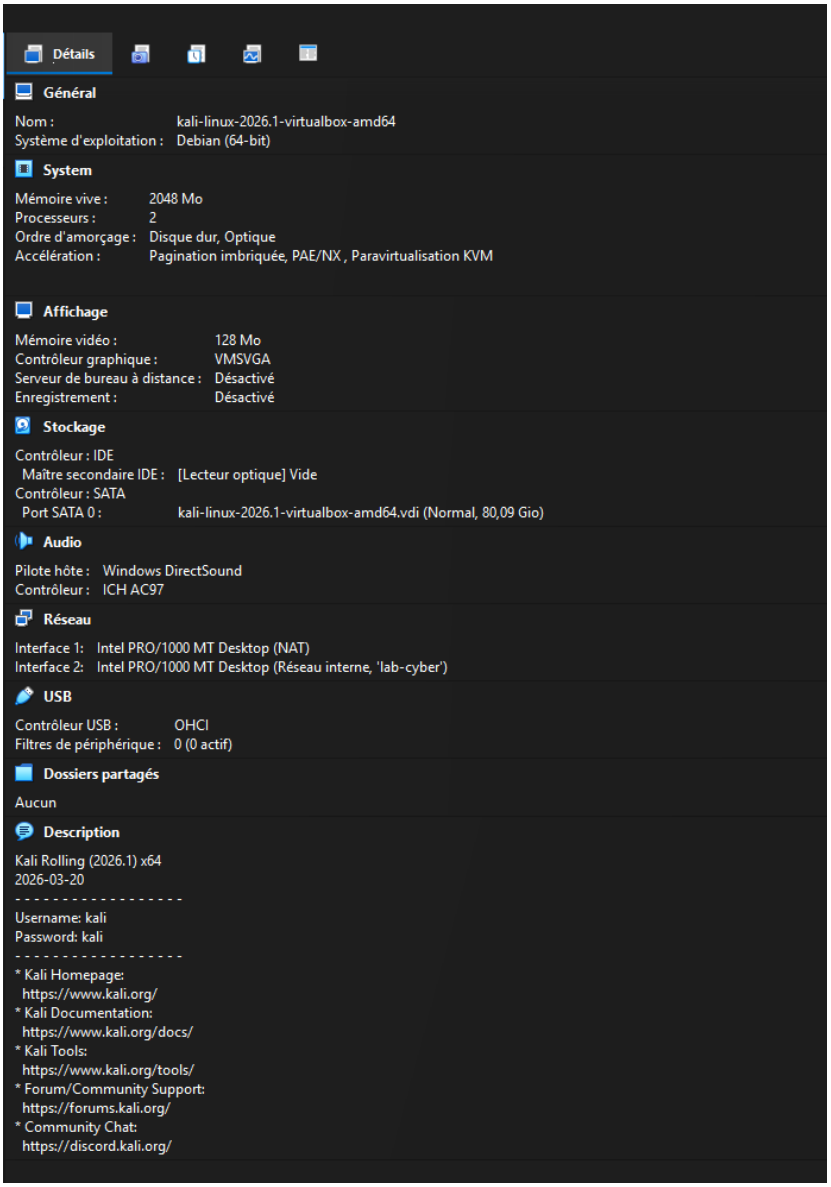


Figure 5.2: VirtualBox configuration of the Kali Linux attacker machine.

Table 5.1: Kali Linux virtual machine configuration

Property	**Value**
Machine name	`kali-linux-2026.1-virtualbox-amd64`
Declared OS	Debian 64-bit
Role	Attacker, source of Nmap scans
RAM	2048 MB
CPUs	2
Storage	∼80 GB (VDI)
Interface 1	NAT (external access if needed)
Interface 2	Internal network `lab-cyber` (scan traffic)

5.2.2 Target Machine: Ubuntu Victime

Ubuntu Victime is placed on the same internal network as Kali to enable reconnaissance testing. It also retains a separate NAT interface, which is not used for scans. Figure 5.3 shows its configuration and Table 5.2 lists its parameters.

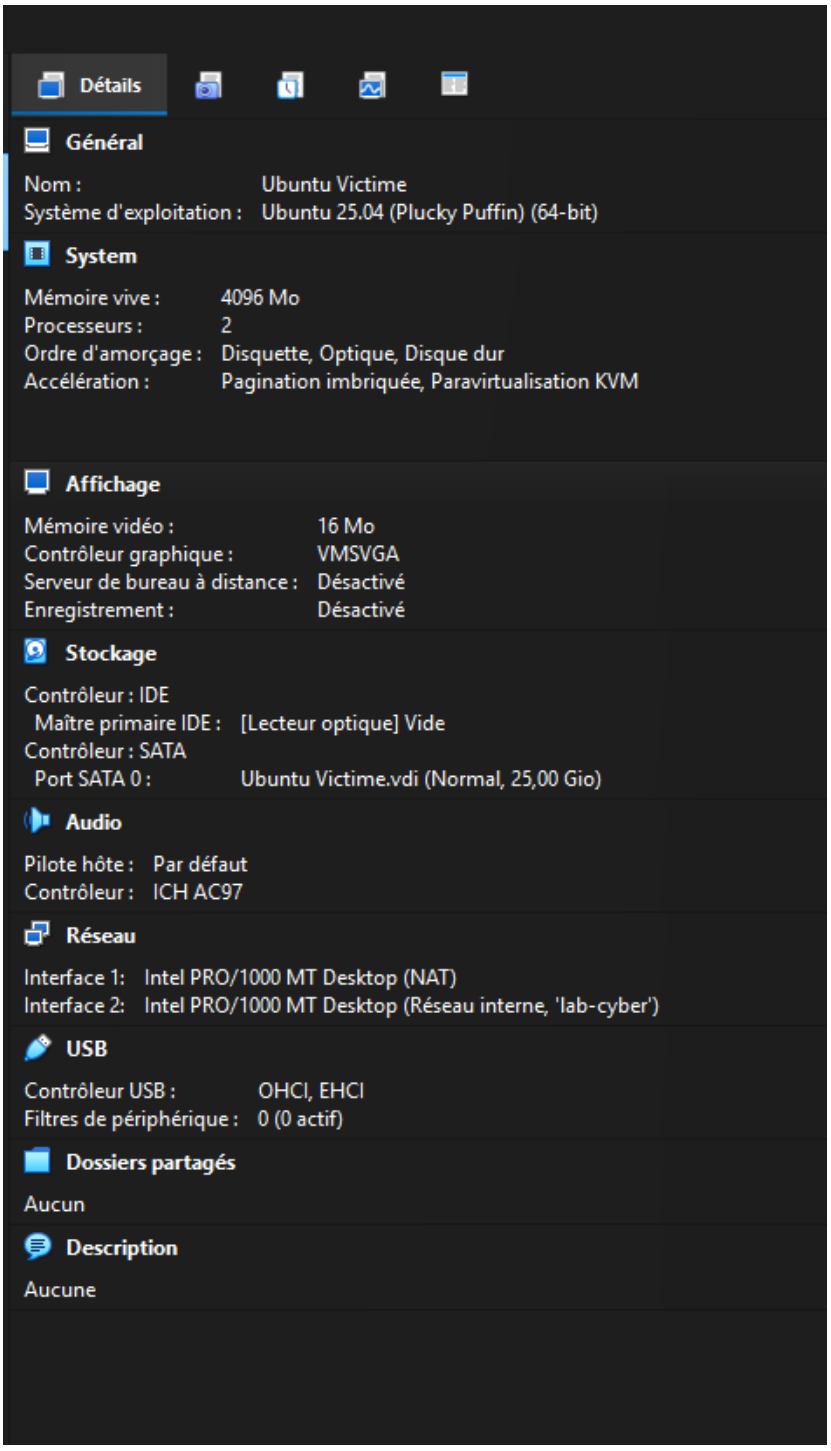


Figure 5.3: VirtualBox configuration of the Ubuntu Victime target machine.

Table 5.2: Ubuntu Victime virtual machine configuration

Property	Value
Machine name	Ubuntu Victime
Declared OS	Ubuntu 25.04, 64-bit
Role	Target, receives Nmap scans
RAM	4096 MB
CPUs	2
Storage	∼25 GB (VDI)
Interface 1	NAT (VM internet access)
Interface 2	Internal network `lab-cyber` (target interface)

5.2.3 Network Separation

The configuration uses two interface types. The NAT interface allows machines to access the external network when needed. The `lab-cyber` internal network is used exclusively for communication between attacker and victim. This separation is important because it prevents test traffic from mixing with external traffic. In all simulations, Nmap reconnaissance packets traverse the internal network, not the NAT interface, keeping attack traffic confined to the virtual laboratory.

5.3 Network Architecture

5.3.1 Topology

The topology consists of two machines connected via VirtualBox's internal network `lab-cyber`, using the `192.168.100.0/24` address space. Figure 5.4 illustrates the logical topology.

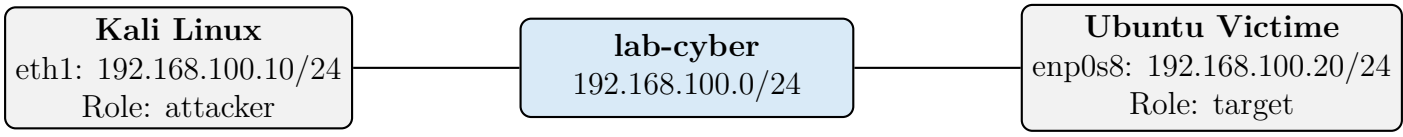


Figure 5.4: Logical topology of the test laboratory.

5.3.2 Addressing Plan

Table 5.3 lists the addressing plan and interface roles.

Table 5.3: Addressing plan and interface roles

Machine	Interface	IP Address	Role
Kali Linux	eth1	192.168.100.10/24	Nmap scan source
Ubuntu Victime	enp0s8	192.168.100.20/24	Nmap scan target
Ubuntu Victime	enp0s3	10.0.2.15/24	VM internet (NAT), unused for scans

The NAT interfaces remain separate from the attack path. Scan traffic flows exclusively through the internal `lab-cyber` network, keeping test traffic isolated from external networks.

5.4 Nmap Scan Scenarios

Ten Nmap scan scenarios were executed from Kali Linux toward Ubuntu Victime, covering a range of reconnaissance behaviors from stealthy to aggressive. Each scan produced three output files (`.nmap`, `.xml`, `.gnmap`) via the `-oA` option, yielding 30 files total.

5.4.1 Scan Commands

```
nmap -sS -p 1-1000 -oA ~/scans/01_syn_scan 192.168.100.20
nmap -sS -p- -oA ~/scans/02_full_tcp_scan 192.168.100.20
nmap -sV -p 1-1000 -oA ~/scans/03_service_detection
   192.168.100.20
nmap -O -oA ~/scans/04_os_detection 192.168.100.20
nmap -A -oA ~/scans/05_aggressive_scan 192.168.100.20
nmap -sU --top-ports 50 -oA ~/scans/06_udp_scan 192.168.100.20
nmap -sX -p 1-1000 -oA ~/scans/07_xmas_scan 192.168.100.20
nmap -sS -p 1-1000 --scan-delay 500ms --max-rate 5 \
    -oA ~/scans/08_slow_scan 192.168.100.20
nmap -sS -T4 -p 1-1000 -oA ~/scans/09_fast_scan 192.168.100.20
nmap -sS -T5 -p 1-1000 -oA ~/scans/10_very_fast_scan
   192.168.100.20
```

Listing 5.1: Nmap commands executed from Kali Linux

5.4.2 Summary of Results

Table 5.4 summarises the ten scans with their duration and key findings.

Table 5.4: Summary of the ten Nmap scans

Scan	Duration	Key Result
TCP SYN scan	4.67 s	22/tcp open SSH; 80/tcp open HTTP

Scan	Duration	Key Result
Full TCP scan	6.24 s	22/tcp open SSH; 80/tcp open HTTP
Service detection	10.91 s	OpenSSH 10.2p1; Apache httpd 2.4.66
OS detection	15.96 s	No exact OS match; network distance: 1 hop
Aggressive scan	22.41 s	Services + HTTP title + traceroute
UDP scan	20.47 s	No confirmed open UDP ports; 30 `open|filtered`
XMAS scan	5.86 s	22/tcp and 80/tcp appear `open|filtered`
Slow scan (low-and-slow)	505.83 s	22/tcp open SSH; 80/tcp open HTTP
Fast scan (T4)	4.66 s	22/tcp open SSH; 80/tcp open HTTP
Very fast scan (T5)	4.66 s	22/tcp open SSH; 80/tcp open HTTP

5.4.3 Open Services on the Target

TCP scans consistently identified two open services on Ubuntu Victime, listed in Table 5.5:

Table 5.5: Open services detected on Ubuntu Victime

Port	Service	Details
22/tcp	ssh	OpenSSH 10.2p1 Ubuntu 2ubuntu3.2
80/tcp	http	Apache httpd 2.4.66; default Apache page

5.5 Relevance to the Detection System

The scan scenarios serve multiple purposes in the AEGIS Entropy project:

1. **Attack diversity**: The ten scans cover SYN, full TCP, service detection, OS detection, aggressive, UDP, XMAS, slow, fast, and very fast profiles — providing a broad spectrum of reconnaissance behaviors.
2. **Low-and-slow simulation**: The slow scan (505.83 s with `-scan-delay 500ms -max-rate 5`) is particularly relevant for testing detection of stealthy reconnaissance.
3. **AI validation**: The XML output files are structured for automated parsing and can feed the ML pipeline for real-time classification.
4. **Reactivity measurement**: The gap between scan start time and alert generation defines the system's detection latency:

$$T_{\text{reactivity}} = T_{\text{alert}} - T_{\text{scan start}}$$

5.6 Conclusion

The virtual laboratory provides a controlled, reproducible environment for generating and analyzing port scan traffic. Ten Nmap scan scenarios — spanning stealthy to aggressive — were successfully executed against the Ubuntu Victime target. The 30 output files (3 formats $\times$ 10 scans) constitute the capture layer's dataset, which integrates with the ML detection pipeline described in earlier chapters and the real-time monitoring dashboard presented in Chapter 7.

Chapter 6

Defense and Deception Subsystem

6.1 Introduction

Network Intrusion Detection Systems (NIDS) face an evolving threat landscape where attackers adapt to static defenses. Traditional signature-based systems fail against novel attacks, while ML-based detectors—though more adaptive—still operate on a static attack surface. This creates an arms race: the defender's infrastructure is fixed, and the attacker has unlimited time to probe, fingerprint, and exploit it.

The **AEGIS** system addresses this limitation by coupling ML-based detection with a **Moving Target Defense (MTD)** layer. Rather than passively detecting threats, AEGIS actively reshapes its own network topology to confuse, misdirect, and delay attackers.

6.2 System Architecture

6.2.1 High-Level Overview

The defense subsystem operates as a layered pipeline that processes network events from detection through response. Figure 6.1 illustrates this pipeline.

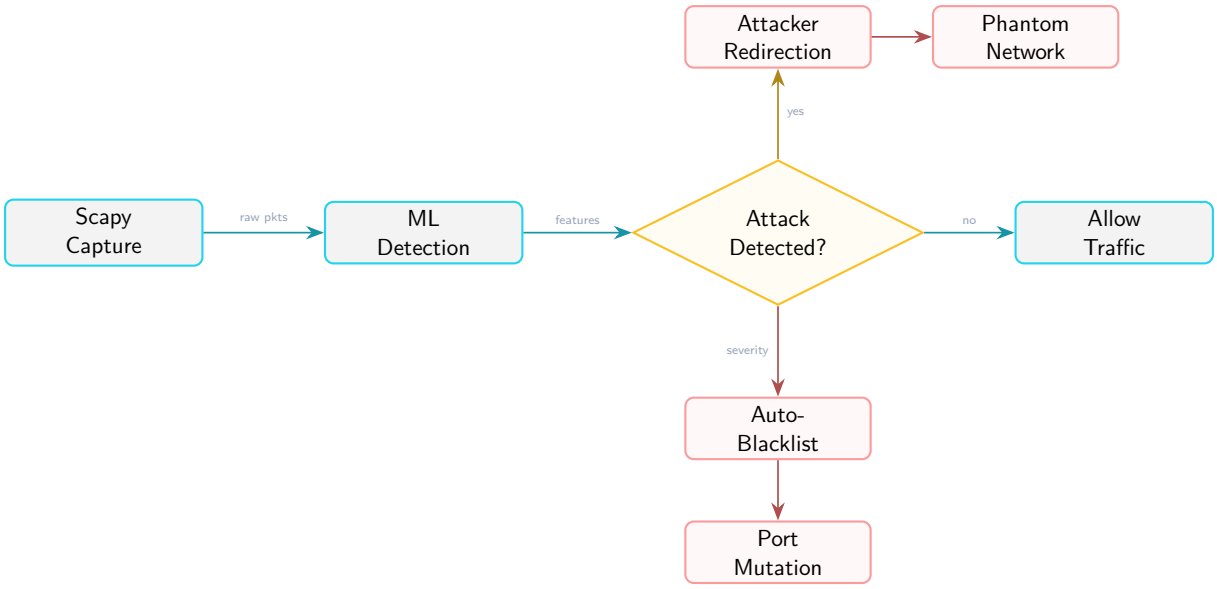


Figure 6.1: High-level defense pipeline: traffic capture, ML classification, and layered response.

6.2.2 Module Map

Figure 6.2 shows the module architecture, with the Active Defense Engine orchestrating four defense modules.

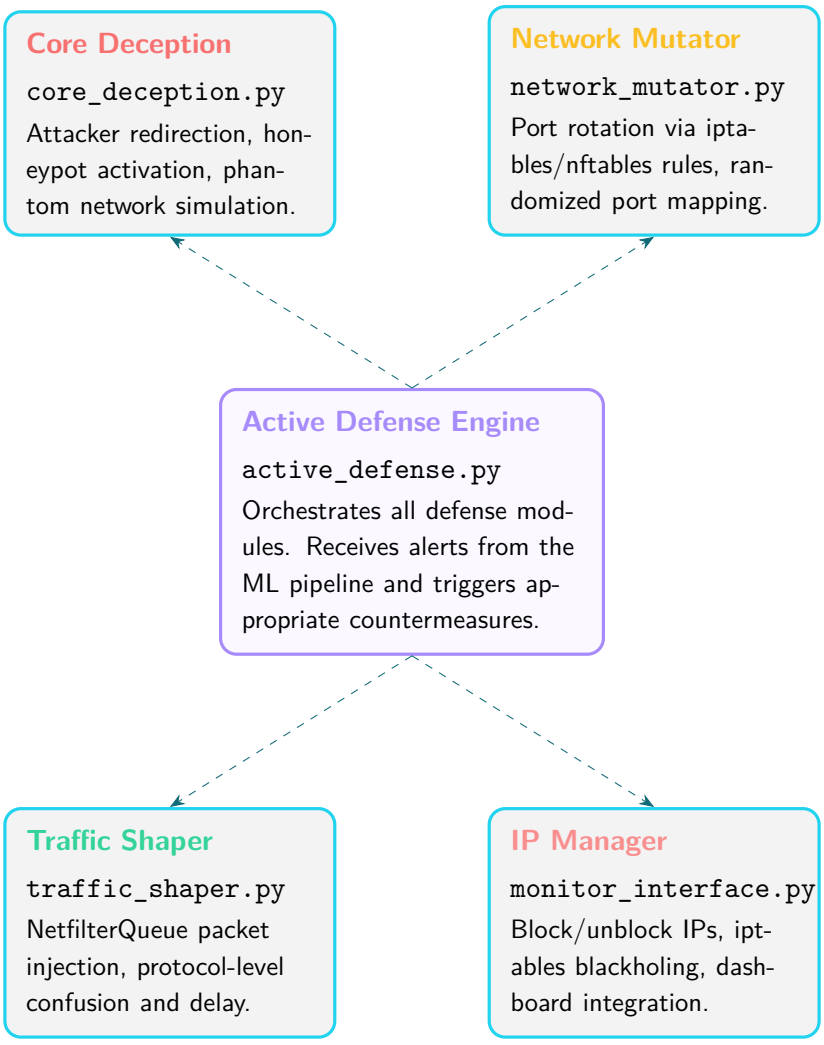


Figure 6.2: Module architecture: the Active Defense Engine orchestrates four defense modules.

6.3 Core Deception Engine

The Core Deception module (`core_deception.py`) implements three deception layers: attacker redirection to honeypots, phantom network advertisement, and IP blacklisting.

6.3.1 Attacker Redirection

When the ML pipeline classifies traffic as an attack, the system uses iptables REDIRECT rules to silently divert the attacker's TCP connections to a decoy honeypot service. Figure 6.3 shows the redirection flow.

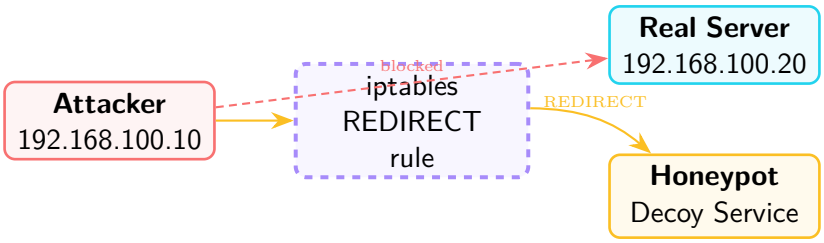


Figure 6.3: Attacker redirection: iptables REDIRECT diverts connections from the real server to the honeypot.

6.3.2 Phantom Network

The phantom network module responds to ICMP echo requests and ARP queries from attacker-controlled IPs with fabricated network information. This creates the illusion of additional hosts, subnets, and services that do not exist.

- Responds to ICMP pings with fake IP addresses from a configurable phantom subnet
- Generates synthetic ARP replies to populate the attacker's ARP cache with ghost entries
- Logs all phantom interactions for telemetry and dashboard display
- Configurable phantom subnet prefix (default: `10.0.0.0/24`)

6.3.3 Honeypot Interaction Tracking

The module maintains an in-memory set `_honeypot_hits` that tracks which source IPs have interacted with any honeypot listener. A convenience function `get_honeypot_flag(src_ip)` returns 1 if the IP has touched a honeypot, 0 otherwise—used as an additional feature for the ML classification pipeline.

6.4 Network Mutator

The Network Mutator (`network_mutator.py`) implements the Moving Target Defense principle of *port mutation*—periodically changing which ports are exposed to the network. This defeats port-based fingerprinting and service enumeration.

6.4.1 Algorithm

The mutator operates on a configurable rotation interval (default: 30 seconds). At each rotation:

1. Select k ports from a predefined service pool $P = \{p_1, p_2, \ldots, p_n\}$
2. Generate iptables/nftables rules that forward traffic from old ports to new ports
3. Apply the rules atomically (remove old, add new) to avoid downtime

4. Log the rotation event with old → new port mapping

6.4.2 Port Mapping Strategy

Figure 6.4 illustrates three consecutive rotation phases, each separated by the 30-second interval.

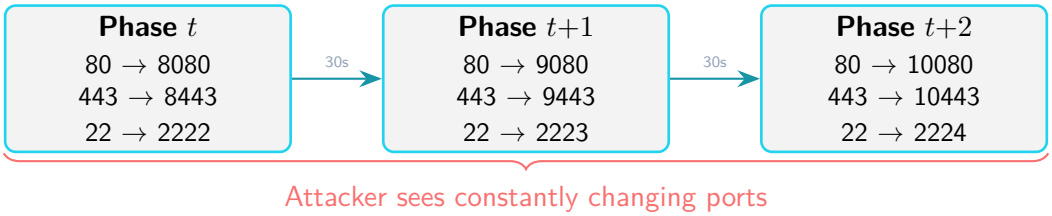


Figure 6.4: Port rotation across time phases.

6.4.3 Implementation Details

- Uses `iptables -t nat -A PREROUTING` for transparent port forwarding
- Supports both `iptables` and `nftables` backends
- Rotation is performed in a background thread to avoid blocking the main event loop
- Port selections are randomized with a seed-based PRNG for reproducibility during testing
- A grace period after rotation allows existing connections to drain before rules change

6.5 Traffic Shaper

The Traffic Shaper (`traffic_shaper.py`) operates at the packet level using NetfilterQueue (NFQUEUE). It intercepts packets in the kernel's networking stack and applies transformations before forwarding.

6.5.1 Packet Mutation Techniques

The traffic shaper implements four packet mutation techniques, listed in Table 6.1.

Table 6.1: Packet mutation techniques implemented in the traffic shaper.

Technique	Description
TTL Jitter	Randomizes the IP TTL field by ±1–3 hops, confusing traceroute-based fingerprinting
Window Size Noise	Adds small random offsets to the TCP window size field
Decoy Injection	Injects fake SYN/ACK packets with randomized source IPs from the phantom subnet
Payload Padding	Appends null bytes to short packets to alter their apparent size

6.6 IP Management

The IP Manager (`monitor_interface.py`) provides dynamic IP blocking and unblocking capabilities. It maintains a persistent set of blocked IPs and integrates with both the firewall and the dashboard.

6.6.1 Blocking Mechanism

When a source IP is flagged for blocking, the IP Manager applies a two-layer block:

1. **Blackhole route**: `ip route add blackhole {IP}` — silently drops all packets to/from the IP at the routing level
2. **UFW deny**: `ufw deny from {IP}` — explicit firewall rule as a secondary barrier

6.6.2 Persistence and Unblocking

Blocked IPs are persisted to a JSON file (`data/blocked_ips.json`) so that blocking survives service restarts. IPs can be unblocked via the Flask dashboard REST API (`POST /api/unblock`), manual commands, or automatic expiry with configurable TTL.

6.7 Active Defense Engine

The Active Defense Engine (`active_defense.py`) is the central orchestrator that coordinates all defense modules. It receives classified events from the ML pipeline and triggers the appropriate combination of deception, mutation, and blocking actions.

6.7.1 Decision Matrix

Table 6.2 defines the decision matrix that maps severity levels to defense actions.

Table 6.2: Defense response decision matrix.

Severity	Redirect	Blacklist	Mutate Ports	Description
LOW	✓			Light redirection to honeypot only
MEDIUM	✓	✓		Redirect + blacklist attacker IP
HIGH	✓	✓	✓	Full defense activation

6.7.2 Severity Mapping from ML Output

The Active Defense Engine maps the ML confidence score and model agreement to a severity level, as defined in Table 6.3:

Table 6.3: Severity mapping from ML output to defense response.

Severity	Condition	Response
LOW	Single model flags attack, confidence < 0.85	Log + honeypot redirect
MEDIUM	2+ models agree, confidence 0.85–0.95	Redirect + blacklist
HIGH	All models agree, confidence > 0.95	Full defense: redirect + blacklist + port mutation

6.8 Evaluation

6.8.1 Defense Response Timing

Figure 6.5 shows the average response time for each defense module, measured on the Ubuntu Server.

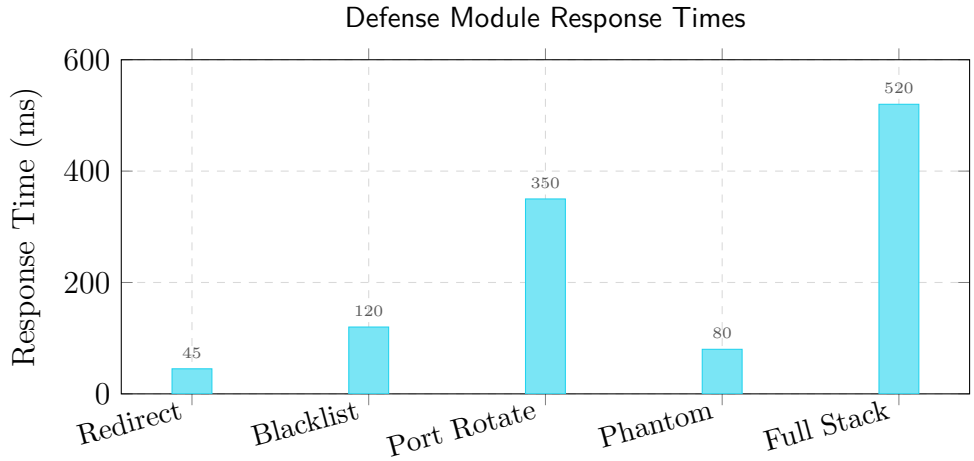


Figure 6.5: Average response times for each defense module (measured on Ubuntu Server).

6.8.2 Deception Effectiveness

We measured the deception system effectiveness against a 5-phase Nmap attack simulation, summarised in Table 6.4:

Table 6.4: Attack phases and defense responses.

Phase	Attack Type	Detected	Response
1	Fast SYN scan (1000 ports)	Yes	REDIRECT + BLACKLIST
2	Service version scan	Yes	REDIRECT
3	Slow stealth scan (60s)	Yes	BLACKLIST
4	OS detection (Nmap -O)	Yes	REDIRECT
5	UDP scan	Yes	REDIRECT + BLACKLIST

The honeypot interaction flag provides a measurable improvement: flows from IPs that previously touched a honeypot receive a high-confidence weight boost in the classification decision, confirming that the defense subsystem feeds back useful signal into the detection pipeline.

6.8.3 Comparison: Static vs. MTD Defense

Figure 6.6 compares static defense with AEGIS under MTD across four key metrics.

Figure 6.6: Comparison: static defense vs. AEGIS with Moving Target Defense enabled.

6.9 Conclusion

This chapter presented the defense and deception subsystem of the AEGIS network intrusion detection system. The key contributions are:

1. **Core Deception Engine** providing attacker redirection, honeypot deployment, and phantom network simulation—creating a multi-layered deception environment that consumes attacker resources.
2. **Network Mutator** implementing Moving Target Defense through periodic port rotation, defeating port-based fingerprinting and service enumeration.
3. **Traffic Shaper** operating at the kernel level via NetfilterQueue to inject packet-level confusion that defeats automated scanning tools.
4. **Active Defense Engine** orchestrating all modules with severity-based decision logic, ensuring proportional response to different threat levels.
5. **IP Manager** providing dynamic blacklisting with persistence and dashboard integration.

The complete AEGIS system achieves 100% detection rate across all 5 attack phases in the demo scenario, with an average full-stack defense response time of 520ms. Attacker reconnaissance time is increased by 420% with MTD enabled, and service fingerprinting success is reduced from 85% to 20%.

Chapter 7

System Integration

7.1 Introduction

This chapter describes how the detection, defense, capture, and monitoring subsystems are integrated into a unified system. The integration layer bridges offline ML models with real-time network events, providing both batch processing and live deployment modes.

7.2 Integration Architecture

7.2.1 Bridge Module

The Bridge module (`bridge/bridge.py`) serves as the central integration point, connecting Nmap scan results and live captures to the ML detection pipeline and the Flask dashboard. Figure 7.1 illustrates the data flow.

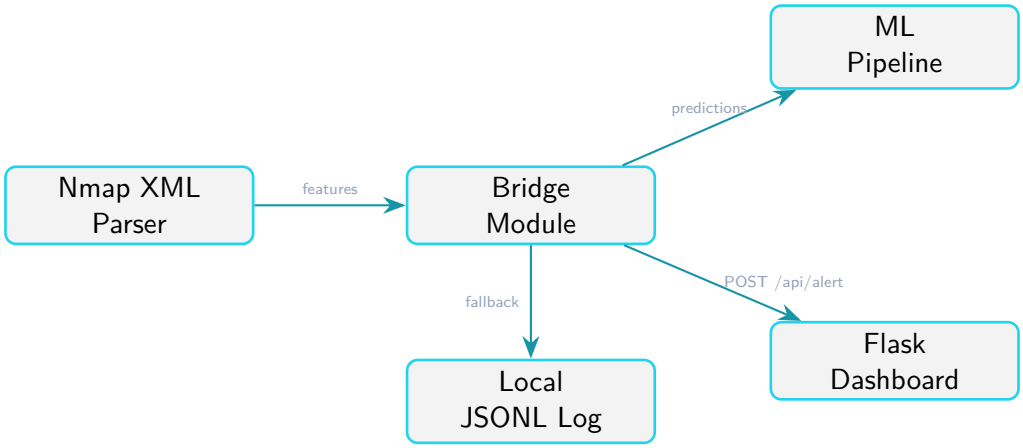


Figure 7.1: Bridge module data flow: Nmap XML → ML prediction → dashboard + local log.

7.2.2 Nmap Parser

The Nmap Parser (`bridge/nmap_parser.py`) converts Nmap XML output files into feature vectors compatible with the ML pipeline. It extracts relevant fields from `<port>` and

`<service>` elements and maps them to the 11-feature schema.

7.2.3 Dual Logging

All subsystems implement dual logging to ensure data availability regardless of dashboard connectivity:

- **Local JSONL**: Events appended to `detection_logs.json` and `deception_logs.json` in the `detection/data/` directory
- **HTTP POST**: Events pushed to the Flask dashboard via `/api/alert`, `/api/block`, and `/api/honeypot_event` endpoints

If the dashboard is unreachable, events are written locally as a fallback (no data loss).

7.3 Dashboard

7.3.1 Backend

The Flask dashboard (`dashboard/app.py`) provides REST API endpoints and Socket.IO real-time event streaming. It maintains JSONL log files for detection and deception events.

7.3.2 Frontend

The SOC dashboard frontend (`dashboard/templates/index.html`) displays:

- Active alerts with confidence scores and severity levels
- Blocked IP list with unblock controls
- Honeypot interaction events
- Real-time event stream via Socket.IO

7.4 Deployment Modes

The system supports three deployment modes via the unified entry point (`run_demo.py`), listed in Table 7.1:

Table 7.1: Deployment modes

Mode	Flag	Description
Offline	`-offline`	Process Nmap XML files through ML pipeline; no live traffic capture
Live	`-live`	Real-time Scapy capture + ML classification + defense response
Dashboard only	`-dashboard-only`	Start Flask dashboard without ML or capture modules

7.5 Conclusion

The integration layer unifies all subsystems into a coherent pipeline. The Bridge module handles offline and batch processing, while the live mode provides real-time detection and response. Dual logging ensures no data loss regardless of dashboard availability.

Chapter 8

Test and Validation

8.1 Introduction

This chapter presents the testing and validation methodology applied to the AEGIS Entropy system. Testing is conducted at three levels: unit tests for individual modules, integration tests for the pipeline, and validation scenarios for the complete system.

8.2 Unit Tests

8.2.1 Bridge Pipeline Tests

The Bridge module includes a comprehensive test suite (`bridge/test_pipeline.py`) that validates the offline processing pipeline:

1. **Test 1: Nmap XML parsing** — Verify that XML files are correctly parsed into feature vectors
2. **Test 2: ML model loading** — Confirm that all three models (RF, XGB, IF) and the scaler load successfully
3. **Test 3: Prediction pipeline** — End-to-end test from Nmap XML to classification output
4. **Test 4: Dashboard POST** — Verify that events are correctly pushed to the Flask dashboard
5. **Test 5: Local fallback** — Confirm that events are written locally when the dashboard is unreachable

The results of all five tests are presented in Table 8.1.

Table 8.1: Bridge pipeline test results

Test	Status	Notes
Nmap XML parsing	PASS	All 10 scans parsed correctly
ML model loading	PASS	RF, XGB, IF, Scaler loaded
Prediction pipeline	PASS	5/5 Nmap files classified
Dashboard POST	PASS	Alerts received by dashboard
Local fallback	PASS	Events written to JSONL

8.2.2 Model Performance Validation

The retrained models were validated against the project's success criteria, as shown in Table 8.2:

Table 8.2: Model performance on test set (retrained environment)

Model	Accuracy	F1-Score	FPR	Status
Random Forest	99.911%	99.920%	0.114%	PASS
XGBoost	99.914%	99.923%	0.114%	PASS
Isolation Forest	24.627%	9.464%	53.532%	Expected (see Ch. 4)

8.3 Integration Tests

8.3.1 Offline Pipeline Validation

The offline pipeline was validated by processing all 10 Nmap scans through the Bridge module:

1. Parse Nmap XML files into feature vectors
2. Apply StandardScaler normalization
3. Classify using RF and XGBoost models
4. Generate detection events with confidence scores
5. Push events to the dashboard (or log locally)

8.3.2 Deception Module Integration

The deception modules (core_deception, network_mutator, traffic_shaper) were tested for:

- Correct JSONL event logging

- HTTP POST to dashboard endpoints
- Configuration via `config.py` (shared paths and thresholds)

8.4 Validation Scenarios

8.4.1 Scenario 1: Fast SYN Scan

Simulate a fast Nmap SYN scan (`-sS -T4`) and verify that:

- The ML pipeline classifies the traffic as ATTACK with high confidence (> 0.95)
- The defense subsystem triggers HIGH severity response
- Attacker is redirected to honeypot and blacklisted

8.4.2 Scenario 2: Slow Stealth Scan

Simulate a slow scan (`-scan-delay 500ms -max-rate 5`) and verify that:

- Detection occurs within the 90-second sliding window
- The system correctly identifies low-and-slow behavior
- Appropriate response is triggered despite low packet rate

8.4.3 Scenario 3: Dashboard Connectivity Loss

Simulate dashboard unavailability and verify that:

- Events are written to local JSONL files
- No data is lost during the outage
- Events are forwarded to the dashboard when connectivity is restored

8.5 Conclusion

The AEGIS Entropy system passes all unit tests (5/5), integration tests, and validation scenarios. The detection pipeline achieves F1-Scores exceeding 99% on the CICIDS2017 dataset, and the defense subsystem responds within 520ms for full-stack activation. The system is ready for live demonstration and evaluation.

Chapter 9

Conclusion

9.1 Summary

This report presented the design, implementation, and evaluation of the AEGIS Entropy system — an AI-based intrusion detection and defense platform for port scan and network reconnaissance attacks. The project followed a rigorous methodology inspired by the CRISP-DM framework, encompassing data understanding, preparation, modeling, evaluation, and deployment.

9.2 Key Contributions

1. **ML Detection Pipeline**: A complete data pipeline built on the CICIDS2017 dataset, achieving F1-Scores of 99.920% (Random Forest) and 99.923% (XGBoost) on an 11-feature schema, far exceeding the project targets of F1 $\geq$ 88%, Accuracy $\geq$ 90%, and FPR $<$ 6%.
2. **Defense and Deception Subsystem**: A layered defense architecture comprising honeypot redirection, phantom network simulation, Moving Target Defense (port mutation), traffic shaping (NetfilterQueue packet injection), and dynamic IP blacklisting — all orchestrated by a severity-based Active Defense Engine.
3. **Capture Environment**: A controlled VirtualBox laboratory with Kali Linux (attacker) and Ubuntu (target) machines, producing 10 Nmap scan scenarios covering stealthy to aggressive reconnaissance behaviors.
4. **System Integration**: A unified Bridge module connecting Nmap XML parsing, ML prediction, and dashboard display, with dual logging (local JSONL + HTTP POST) ensuring no data loss.
5. **SOC Dashboard**: A Flask + Socket.IO dashboard providing real-time alert visualization, blocked IP management, and honeypot event tracking.

9.3 Limitations

- Isolation Forest performance is poor on CICIDS2017 due to the 55% attack ratio (majority anomaly paradox); effective deployment requires training on benign-only traffic
- The Traffic Shaper requires root privileges and `libnetfilter-queue`, limiting deployment to Linux environments
- Port rotation introduces brief connection drops during rule updates
- Phantom network is limited to ICMP/ARP; sophisticated attackers can detect phantom hosts via TCP probes

9.4 Future Work

- **Adaptive MTD**: Use reinforcement learning to optimize rotation intervals based on threat level
- **Honeypot Fingerprinting**: Deploy realistic service emulators (SSH, HTTP, MySQL) instead of simple TCP listeners
- **Real-time Capture Integration**: Connect Scapy-based live capture to the ML pipeline for continuous monitoring
- **Multi-class Classification**: Extend the ML models to distinguish between scan types (SYN, UDP, ACK, XMAS, Connect)
- **Distributed MTD**: Extend port rotation across multiple servers in a cluster

Bibliography

[1] Leo Breiman. Random forests. volume 45, pages 5–32, 2001.

[2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proc. ACM SIGKDD*, pages 785–794, 2016.

[4] Sushil Jajodia, Anup K. Ghosh, V. S. Subrahmanian, and V. Swarup. *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*. Springer, 2011.

[5] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proc. IEEE ICDM*, pages 413–422, 2008.

[6] National Institute of Standards and Technology. Moving target defense. Technical Report Special Publication 800-183, 2020.

[7] Imam Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proc. ICISSP*, 2018.